DATA 260 – HW 9

7 - Community Sports League Fixtures

SJSU ID : 020053319 SRI PRAJEET VILLYAL

GitHub repository: https://github.com/villyalsriprajeet-cmd/data260-3319

Tagged commit hash (hw9): e3d3420b61b306342dc6f1fc1b3bfd1ec1acbf05

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b through Ollama

Section 0 - Personal Configuration and Domain

Value	Mine
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7 - Community sports league fixtures

Redis runs as Redis Stack 7.4.0-v8 (Redis with vector search) in Docker, container s3319_redis. Every Redis key starts with s3319:. HW9 code is in code/hw9/ and the results are in reports/hw09/.



Redis, Kafka, worker and API containers running.

Part 1: Redis View Counter

My entity is a fixture. I added one method, increment_fixture_views(fixture_id), to FixtureCache. Every access runs INCR on s3319:views:fixture:{id}. When count % 10 == 0 the count is written to fixtures.views in MySQL and "Synced views to database" is logged. The method returns the current count.

Code

```yaml
    ▷Run Service
  redis:  # HW9: Redis Stack (Redis + vector search) for the view counter and the semantic cache
    image: redis/redis-stack-server:7.4.0-v8
    container_name: s3319_redis
    environment:
      REDIS_ARGS: "--appendonly yes"  # keep the cache and counters across restarts
    volumes:
      - s3319_redis_data:/data
    ports:
      - "127.0.0.1:6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10
    restart: unless-stopped

volumes:
  s3319_redis_data:

```

docker-compose.yml: the Redis service.

```python
code > hw9 > fixture_cache.py > ...
class FixtureCache:
    def increment_fixture_views(self, fixture_id, on_count=None):
        key = self.views_key(fixture_id)
        if not self.redis_client.exists(key):  # first view since Redis started: begin from the saved count
            saved = self.db_views(fixture_id)
            if saved is None:
                raise ValueError(f"Fixture {fixture_id} not found")
            self.redis_client.set(key, saved, nx=True)  # nx: never overwrite a count another process set
        count = self.redis_client.incr(key)  # atomic +1 in Redis, no database query
        if on_count:
            on_count(count)  # lets the caller print the access before the sync message
        if count % SYNC_EVERY == 0:
            self.sync_views_to_db(fixture_id, count)
        return count

    def sync_views_to_db(self, fixture_id, count):
        start = time.perf_counter()
        with self.engine.begin() as conn:
            conn.execute(
                # updated_at = updated_at keeps the row's data version, a view is not a data change
                text("UPDATE fixtures SET views = :views, updated_at = updated_at WHERE id = :id"),
                {"views": count, "id": fixture_id},
            )
        ms = (time.perf_counter() - start) * 1000
        print(f"✓ Synced views to database (fixture {fixture_id}, views = {count}, {ms:.1f} ms)", flush=True)

```

fixture_cache.py: increment_fixture_views.

code > api > sql > 005_hw09_views.sql

```sql
-- HW9 Part 1: view count of each fixture, written by the Redis counter every 10 views
USE s3319_rel;
ALTER TABLE fixtures ADD COLUMN views INT NOT NULL DEFAULT 0 AFTER tickets_available;

```

005_hw09_views.sql: views column.

Output

```
(data260) villyalsriprajeet@Mac data260-3319 % python -m code.hw9.p1_counter_test
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - HW9 Part 1 fixture view counter
Fixture 42 before: {'id': 42, 'fixture_code': 'FX-00042', 'views': 0, 'updated_at': datetime.datetime(2026, 10, 1, 17, 14, 36)}
Redis key: s3319:views:fixture:42

=== Testing Counter ===

Access #1: Total = 1
Access #2: Total = 2
Access #3: Total = 3
Access #4: Total = 4
Access #5: Total = 5
Access #6: Total = 6
Access #7: Total = 7
Access #8: Total = 8
Access #9: Total = 9
Access #10: Total = 10
✓ Synced views to database (fixture 42, views = 10, 14.3 ms)
Access #11: Total = 11
Access #12: Total = 12
Access #13: Total = 13
Access #14: Total = 14
Access #15: Total = 15
Access #16: Total = 16
Access #17: Total = 17
Access #18: Total = 18
Access #19: Total = 19
Access #20: Total = 20
✓ Synced views to database (fixture 42, views = 20, 2.5 ms)
Access #21: Total = 21
Access #22: Total = 22
Access #23: Total = 23
Access #24: Total = 24
Access #25: Total = 25

Redis  s3319:views:fixture:42 = 25
MySQL  fixtures.views = 20 (synced at every 10 views)
MySQL  updated_at unchanged by the syncs: True
Average time per access: 2.74 ms over 25 accesses
(data260) villyalsriprajeet@Mac data260-3319 %
```

25 accesses: synced at 10 and 20.

```
(data260) villyalsriprajeet@Mac data260-3319 % echo "Sri Prajeet Villyal - HW9 Part 1 Redis vs MySQL"; docker exec s3319_redis redis-cli GET s3319:views:fixture:42; docker exec s3319_redis redis-cli --scan --
pattern 's3319:*'; mysql -u root -p -e "SELECT id, fixture_code, views FROM s3319_rel.fixtures WHERE id = 42;"
Sri Prajeet Villyal - HW9 Part 1 Redis vs MySQL
25
s3319:views:fixture:42
Enter password:
+----+--------------+-------+
| id | fixture_code | views |
+----+--------------+-------+
| 42 | FX-00042     |    20 |
+----+--------------+-------+
(data260) villyalsriprajeet@Mac data260-3319 %
```

Redis 25, MySQL 20.

The count went from 1 to 25 and was written to MySQL twice, after access 10 and 20. Redis ends at 25 and MySQL at 20, the last synced value.

Part 2: Semantic Response Cache

The cache sits in front of the HW8 Planner, Writer and Reviewer. Each question is embedded with all-MiniLM-L6-v2 (384 dims) and searched in the Redis vector index s3319:idx:qa with cosine similarity. Threshold 0.85.

- Hit: return the stored final answer with is_cached=true and the similarity. The agents do not run. CACHE_HIT is logged with the similarity.
- Miss: CACHE_MISS is logged and the question goes through the HW8 agents. The query, embedding, final answer and data-version marker (fixture code @ updated_at) are stored. Returns is_cached=false.
- Latency: time.perf_counter() before and after the embedding, the vector search and the agent run.

Code

```
47      def ensure_index(self):
48          try:
49              self.r.ft(INDEX).info()
50              return  # already there
51          except ResponseError:
52              pass
53          schema = (
54              TextField("query"),
55              TagField("fixture_codes"),
56              TagField("data_version"),
57              NumericField("created_at"),
58              VectorField("embedding", "FLAT", {"TYPE": "FLOAT32", "DIM": DIM, "DISTANCE_METRIC": "COSINE"}),
59          )
60          definition = IndexDefinition(prefix=[DOC_PREFIX], index_type=IndexType.HASH)
61          self.r.ft(INDEX).create_index(schema, definition=definition)
62
63      def key_for(self, query):
64          return DOC_PREFIX + hashlib.sha1(normalize(query).lower().encode()).hexdigest()[:16]
65
66      def nearest(self, vector):
67          q = Query("*=>[KNN 1 @embedding $vec AS dist]").sort_by("dist").return_fields(*RETURN_FIELDS).dialect(2)
68          res = self.r.ft(INDEX).search(q, query_params={"vec": vector.astype(np.float32).tobytes()})
69          if not res.docs:
70              return None  # empty cache
71          doc = res.docs[0]
72          entry = {name: getattr(doc, name, "") for name in RETURN_FIELDS if name != "dist"}
73          entry["key"] = doc.id
74          entry["similarity"] = 1.0 - float(doc.dist)  # Redis returns cosine distance = 1 - cosine similarity
75          return entry
76
77      def lookup(self, query, vector=None):
78          vector = self.embed_fn(normalize(query)) if vector is None else vector
79          best = self.nearest(vector)
80          hit = best is not None and best["similarity"] >= self.threshold
81          return hit, best, vector
```

semantic_cache.py: cosine vector index and lookup.

```python
    async def handle_query(self, query, use_cache=True):
        started_at = datetime.now(timezone.utc).isoformat(timespec="milliseconds")
        t0 = time.perf_counter()
        query = normalize(query)
        vector = self.cache.embed_fn(query)  # step 1: embed the question
        t1 = time.perf_counter()
        hit, best, vector = self.cache.lookup(query, vector)  # step 2: nearest cached question in Redis
        t2 = time.perf_counter()
        embed_ms, search_ms, lookup_ms = (t1 - t0) * 1000, (t2 - t1) * 1000, (t2 - t0) * 1000
        best_sim = round(best["similarity"], 4) if best else None
        if use_cache and hit and self.check_version:
            stored_codes = [c for c in best["fixture_codes"].split(",") if c]
            now_version = self.current_version(stored_codes)
            if now_version != best["data_version"]:  # the record changed after the answer was cached
                self.cache.delete(best["key"])
                qlog("STALE_INVALIDATED", similarity=best_sim, cached=best["data_version"], current=now_version)
                hit = False
        if use_cache and hit:
            latency_ms = (time.perf_counter() - t0) * 1000
            qlog("CACHE_HIT", similarity=best_sim, matched_query=best["query"], latency_ms=round(latency_ms, 1))
            return {
                "query": query,
                "answer": best["answer"],
                "is_cached": True,
                "similarity": best_sim,
                "matched_query": best["query"],
                "data_version": best["data_version"],
                "latency_ms": round(latency_ms, 1),
                "lookup_ms": round(lookup_ms, 1),
                "embed_ms": round(embed_ms, 1),
                "search_ms": round(search_ms, 1),
                "agent_ms": 0.0,
                "started_at": started_at,
                "finished_at": datetime.now(timezone.utc).isoformat(timespec="milliseconds"),
            }
        cid = str(uuid.uuid4())
```

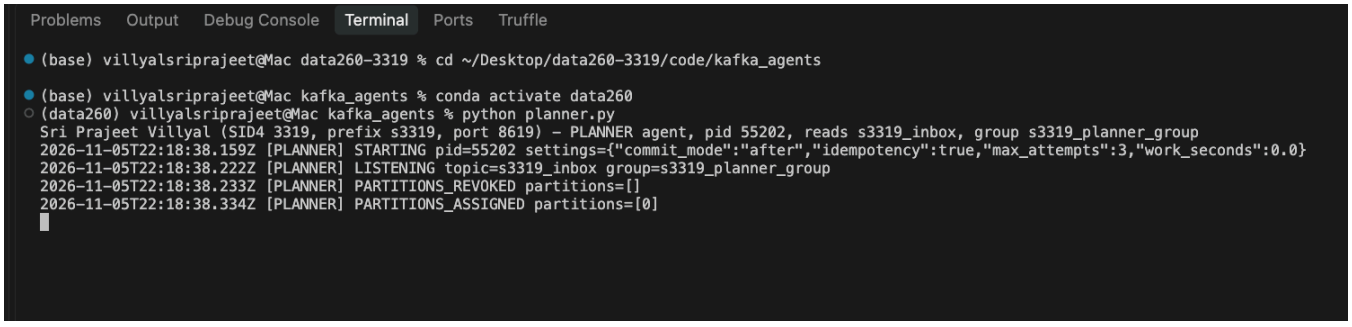
qa_service.py: cache hit, is_cached=True.

```python
        cid = str(uuid.uuid4())
        qlog("CACHE_MISS", cid, best_similarity=best_sim, threshold=self.cache.threshold, query=query)
        codes = fixture_codes(query)
        rows = self.load_rows(FACTS_SQL, codes)
        version = version_of(codes, rows)
        t_agent = time.perf_counter()
        final = await self.agents(query + facts_text(codes, rows), cid)
        agent_ms = (time.perf_counter() - t_agent) * 1000
        answer = str(final.get("answer", "")).strip()
        if use_cache and answer:
            key = self.cache.store(query, vector, answer, version, codes, correlation_id=cid)
            qlog("CACHE_STORED", cid, key=key, data_version=version)
        latency_ms = (time.perf_counter() - t0) * 1000
        return {
            "query": query,
            "answer": answer,
            "is_cached": False,
            "similarity": best_sim,
            "matched_query": best["query"] if best else None,
            "data_version": version,
            "latency_ms": round(latency_ms, 1),
            "lookup_ms": round(lookup_ms, 1),
            "embed_ms": round(embed_ms, 1),
            "search_ms": round(search_ms, 1),
            "agent_ms": round(agent_ms, 1),
            "correlation_id": cid,
            "review_status": final.get("status"),
            "started_at": started_at,
            "finished_at": datetime.now(timezone.utc).isoformat(timespec="milliseconds"),
        }

```

qa_service.py: cache miss, HW8 agents, store.

HW8 agents running



Planner.



Writer.

```
Problems  Output  Debug Console  Terminal  Ports  Truffle

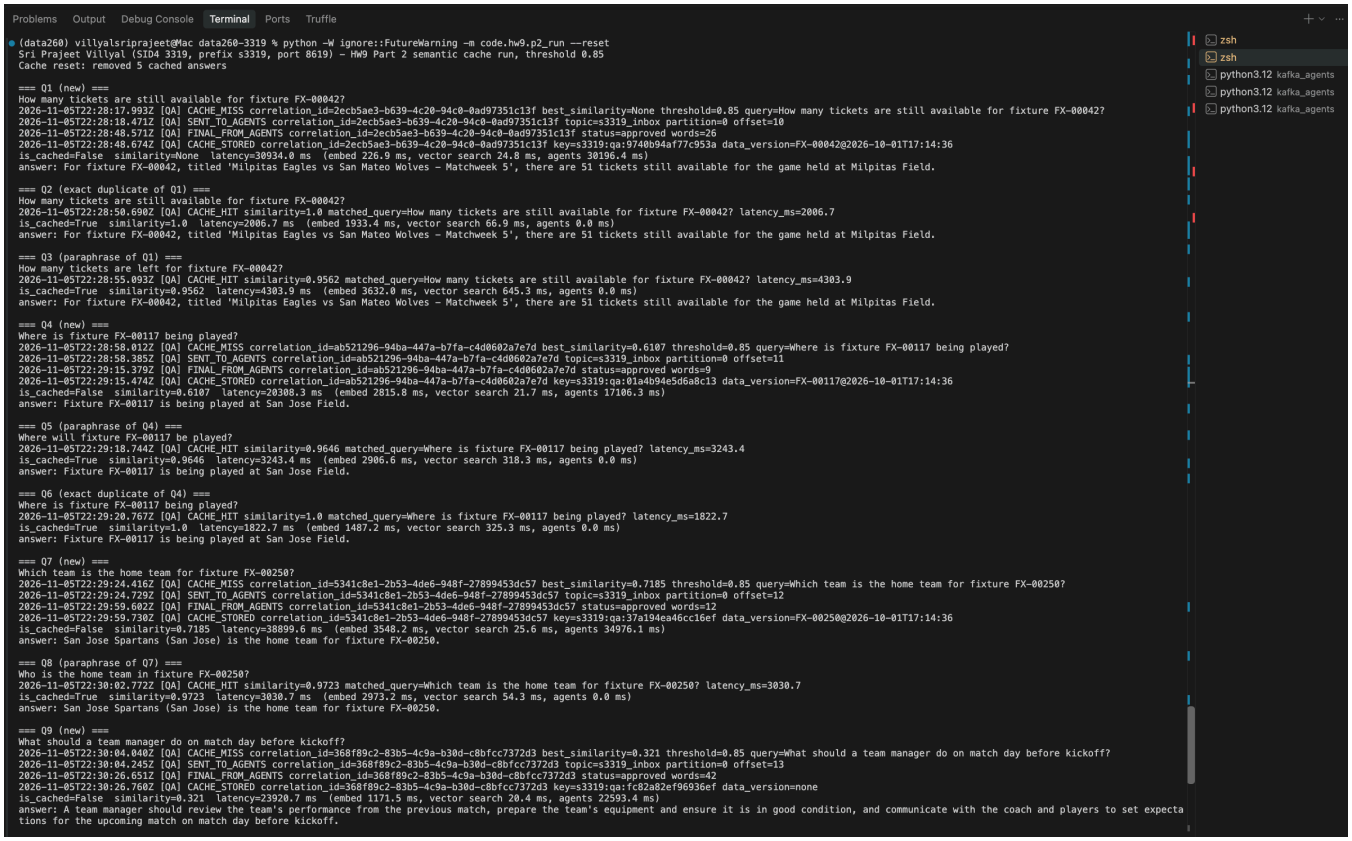
(base) villyalsriprajeet@Mac data260-3319 % cd ~/Desktop/data260-3319/code/kafka_agents

(base) villyalsriprajeet@Mac kafka_agents % conda activate data260
(data260) villyalsriprajeet@Mac kafka_agents % python reviewer.py
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - REVIEWER agent, pid 56586, reads s3319_drafts, group s3319_reviewer_group
2026-11-05T22:19:20.613Z [REVIEWER] STARTING pid=56586 settings={"commit_mode":"after","idempotency":true,"max_attempts":3,"work_seconds":0.0}
2026-11-05T22:19:20.659Z [REVIEWER] LISTENING topic=s3319_drafts group=s3319_reviewer_group
2026-11-05T22:19:20.669Z [REVIEWER] PARTITIONS_REVOKED partitions=[]
2026-11-05T22:19:20.684Z [REVIEWER] PARTITIONS_ASSIGNED partitions=[0]
```

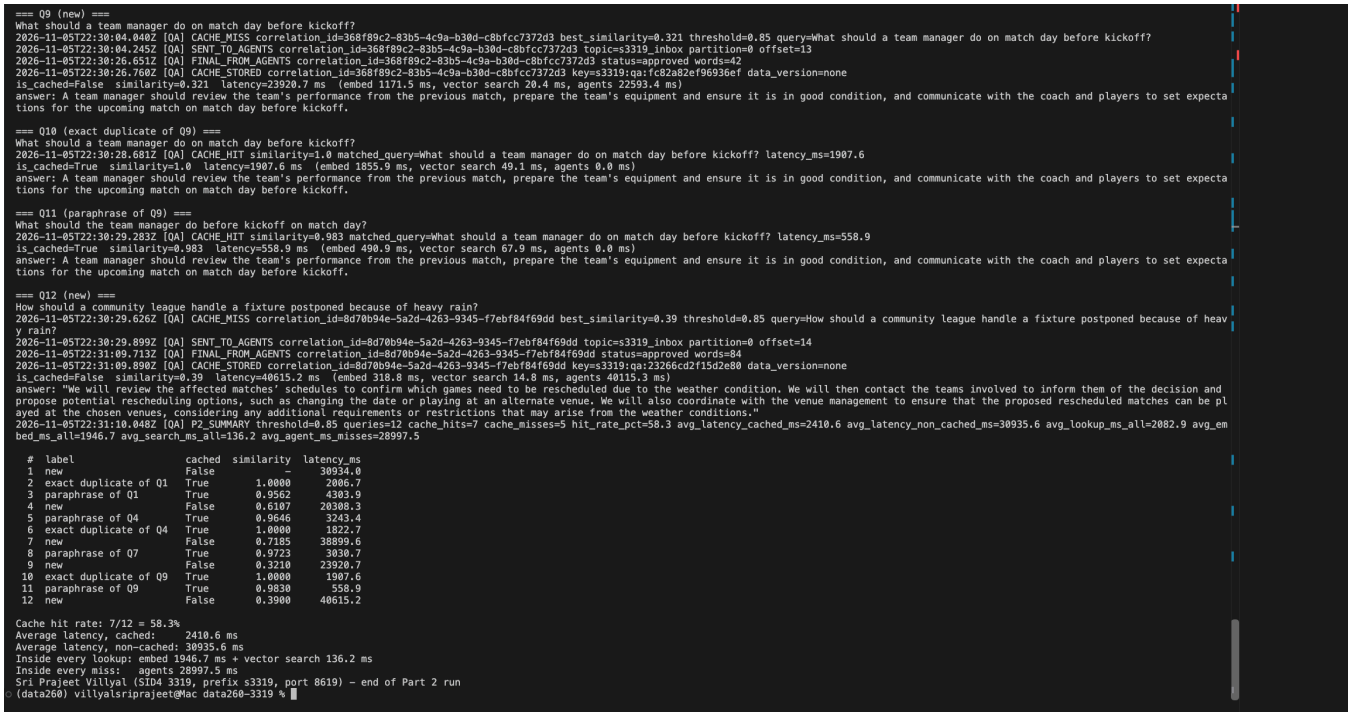
Reviewer.

12 test queries

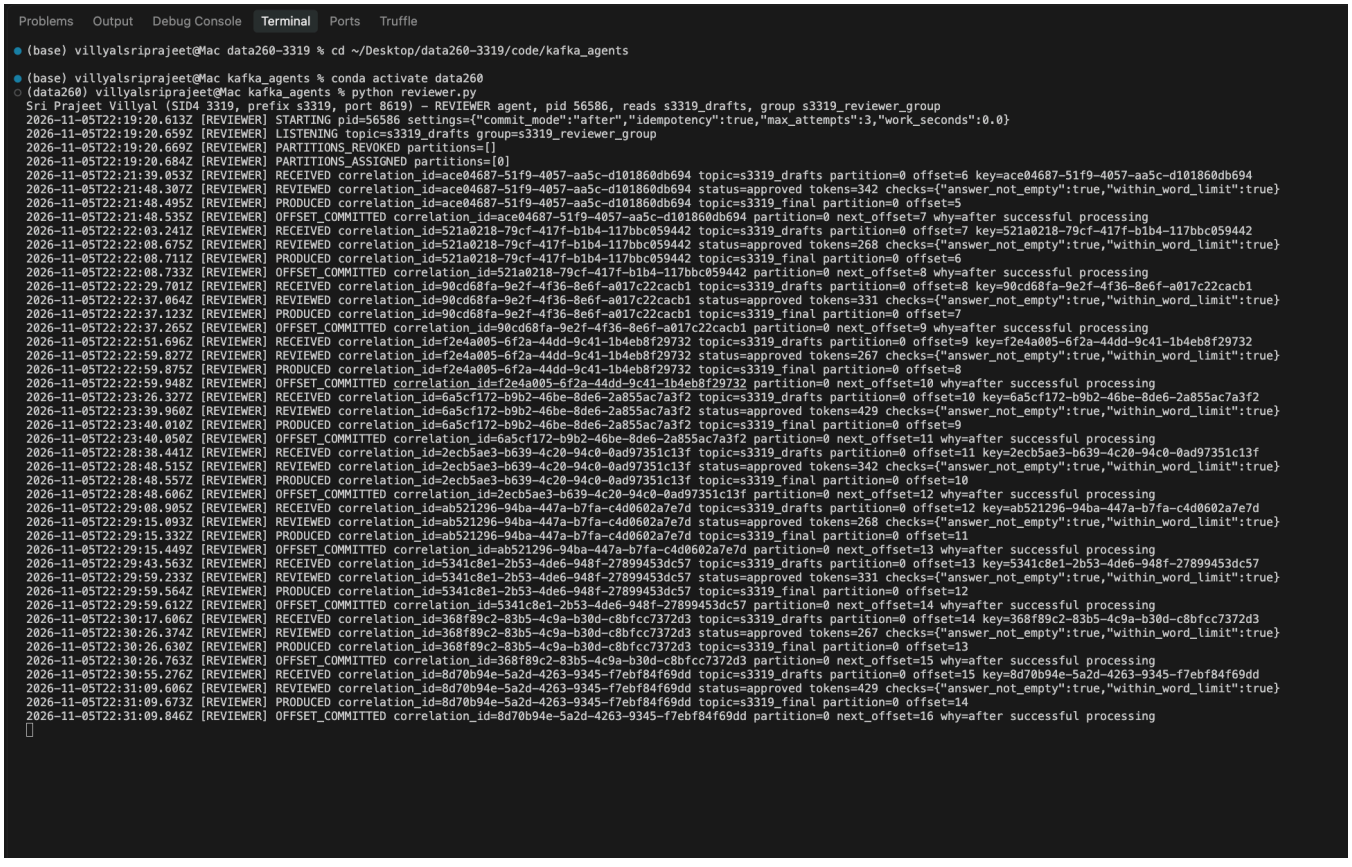
5 new questions, 3 exact duplicates and 4 paraphrases, starting from an empty cache.



Queries 1 to 9.



Queries 9 to 12 and the summary.



Agents answering the misses.

#	Query type	is_cached	Similarity	Latency (ms)
1	new	False	-	30,934.0
2	exact duplicate of Q1	True	1.0000	2,006.7
3	paraphrase of Q1	True	0.9562	4,303.9
4	new	False	0.6107	20,308.3
5	paraphrase of Q4	True	0.9646	3,243.4
6	exact duplicate of Q4	True	1.0000	1,822.7
7	new	False	0.7185	38,899.6
8	paraphrase of Q7	True	0.9723	3,030.7
9	new	False	0.3210	23,920.7
10	exact duplicate of Q9	True	1.0000	1,907.6
11	paraphrase of Q9	True	0.9830	558.9
12	new	False	0.3900	40,615.2

Results

Measure	Measured
Cache hit rate	7/12 = 58.3%
Average latency, cached	2,410.6 ms
Average latency, non-cached	30,935.6 ms

All duplicates and paraphrases hit and all new questions missed. A cached answer was about 13 times faster. Most of the cached time was the embedding on the CPU (1,946.7 ms on average) while Ollama and the three agents were also running; the Redis search itself took 136.2 ms.

Part 3: Threshold Economics and Stale Answers

3.1 Labeled test set

cache_cases.yaml has 10 seed questions that are cached first and 40 labeled cases: 10 exact duplicates and 10 paraphrases (should hit), 10 superficially similar questions with a different answer and 10 unrelated questions (should miss). It was committed in 4473d65, before the sweep. No labels were changed.

reports > hw09 > ! cache_cases.yaml

```yaml
# HW9 Part 3.1 — labeled cache test set, Domain 7 (community sports league fixtures)
# Written and committed before any threshold sweep. Labels come from the meaning of each question,
# not from similarity scores. Do not edit after the sweep: any labeling mistake is written up in
# reports/hw09/LABEL_NOTES.md instead.
#
# seeds: the 10 questions that are in the cache before the sweep (answered once by the HW8 agents)
# cases: 40 incoming questions, 10 per category
#   exact_duplicate    same text as a seed                       expected: hit  (ref = that seed)
#   paraphrase         same meaning and same answer as a seed    expected: hit  (ref = that seed)
#   similar_different  looks like a seed but has another answer  expected: miss (ref = the look-alike seed)
#   unrelated          not about any seed                        expected: miss (ref = null)
# A hit is correct only if it returns the ref seed of a case that expects a hit. Any other hit is a false hit.

seeds:
  - {id: S01, query: "How many tickets are still available for fixture FX-00042?"}
  - {id: S02, query: "Where is fixture FX-00117 being played?"}
  - {id: S03, query: "Which team is the home team for fixture FX-00250?"}
  - {id: S04, query: "What is the title of fixture FX-00300?"}
  - {id: S05, query: "Is fixture FX-00512 sold out?"}
  - {id: S06, query: "Which city is the home team of fixture FX-00777 from?"}
  - {id: S07, query: "What should a team manager do on match day before kickoff?"}
  - {id: S08, query: "How should a community league handle a fixture postponed because of heavy rain?"}
  - {id: S09, query: "How many points does a team get for a win in the league?"}
  - {id: S10, query: "What should players bring to an away game?"}

cases:
  # exact duplicates — same text as the seed, should hit
  - {id: E01, category: exact_duplicate, expected: hit, ref: S01, query: "How many tickets are still available for fixture FX-00042?", why: "identical text"}
  - {id: E02, category: exact_duplicate, expected: hit, ref: S02, query: "Where is fixture FX-00117 being played?", why: "identical text"}
  - {id: E03, category: exact_duplicate, expected: hit, ref: S03, query: "Which team is the home team for fixture FX-00250?", why: "identical text"}
  - {id: E04, category: exact_duplicate, expected: hit, ref: S04, query: "What is the title of fixture FX-00300?", why: "identical text"}
  - {id: E05, category: exact_duplicate, expected: hit, ref: S05, query: "Is fixture FX-00512 sold out?", why: "identical text"}
  - {id: E06, category: exact_duplicate, expected: hit, ref: S06, query: "Which city is the home team of fixture FX-00777 from?", why: "identical text"}
  - {id: E07, category: exact_duplicate, expected: hit, ref: S07, query: "What should a team manager do on match day before kickoff?", why: "identical text"}
  - {id: E08, category: exact_duplicate, expected: hit, ref: S08, query: "How should a community league handle a fixture postponed because of heavy rain?", why: "identical text"}
  - {id: E09, category: exact_duplicate, expected: hit, ref: S09, query: "How many points does a team get for a win in the league?", why: "identical text"}
  - {id: E10, category: exact_duplicate, expected: hit, ref: S10, query: "What should players bring to an away game?", why: "identical text"}

```

cache_cases.yaml.

```
(data260) villyalsriprajeet@Mac data260-3319 % python -m code.hw9.p3_cases
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - HW9 Part 3.1 cache_cases.yaml check
File: reports/hw09/cache_cases.yaml
Seeds (cached before the sweep): 10
  exact_duplicate    expected hit   10 cases
  paraphrase         expected hit   10 cases
  similar_different  expected miss  10 cases
  unrelated          expected miss  10 cases
Total cases: 40  - all checks passed

E01  exact_duplicate    hit  ref=S01   How many tickets are still available for fixture FX-00042?
E02  exact_duplicate    hit  ref=S02   Where is fixture FX-00117 being played?
E03  exact_duplicate    hit  ref=S03   Which team is the home team for fixture FX-00250?
E04  exact_duplicate    hit  ref=S04   What is the title of fixture FX-00300?
E05  exact_duplicate    hit  ref=S05   Is fixture FX-00512 sold out?
E06  exact_duplicate    hit  ref=S06   Which city is the home team of fixture FX-00777 from?
E07  exact_duplicate    hit  ref=S07   What should a team manager do on match day before kickoff?
E08  exact_duplicate    hit  ref=S08   How should a community league handle a fixture postponed because of heavy rain?
E09  exact_duplicate    hit  ref=S09   How many points does a team get for a win in the league?
E10  exact_duplicate    hit  ref=S10   What should players bring to an away game?
P01  paraphrase         hit  ref=S01   How many tickets are left for fixture FX-00042?
P02  paraphrase         hit  ref=S02   What venue is fixture FX-00117 played at?
P03  paraphrase         hit  ref=S03   Who is the home side in fixture FX-00250?
P04  paraphrase         hit  ref=S04   What is fixture FX-00300 called?
P05  paraphrase         hit  ref=S05   Are there any tickets left for fixture FX-00512, or is it sold out?
P06  paraphrase         hit  ref=S06   What city does the home team of fixture FX-00777 come from?
P07  paraphrase         hit  ref=S07   Before kickoff on match day, what should the team manager do?
P08  paraphrase         hit  ref=S08   What should the league do when heavy rain postpones a fixture?
P09  paraphrase         hit  ref=S09   How many league points is a win worth?
P10  paraphrase         hit  ref=S10   What items should players pack for an away match?
D01  similar_different  miss ref=S01   How many tickets are still available for fixture FX-00043?
D02  similar_different  miss ref=S02   Where is fixture FX-00118 being played?
D03  similar_different  miss ref=S03   Which team is the away team for fixture FX-00250?
D04  similar_different  miss ref=S04   What is the venue of fixture FX-00300?
D05  similar_different  miss ref=S05   Is fixture FX-00513 sold out?
D06  similar_different  miss ref=S06   Which city is the home team of fixture FX-00778 from?
D07  similar_different  miss ref=S07   What should a team manager do on match day after the final whistle?
D08  similar_different  miss ref=S08   How should a community league handle a fixture abandoned because of a floodlight failure?
D09  similar_different  miss ref=S09   How many points does a team get for a draw in the league?
D10  similar_different  miss ref=S10   What should referees bring to an away game?
U01  unrelated          miss ref=None  How do I reset my password on the league website?
U02  unrelated          miss ref=None  What is the weather forecast for Saturday?
U03  unrelated          miss ref=None  How do I register a new player for next season?
U04  unrelated          miss ref=None  Who won the 2023-24 Premier League?
U05  unrelated          miss ref=None  What are the opening hours of the league office?
U06  unrelated          miss ref=None  How much does it cost to rent a pitch for an hour?
U07  unrelated          miss ref=None  Can I bring my dog to a match?
U08  unrelated          miss ref=None  How do I become a volunteer referee?
U09  unrelated          miss ref=None  What is the offside rule?
U10  unrelated          miss ref=None  Where can I buy the official league kit?
(data260) villyalsriprajeet@Mac data260-3319 %
```

40 cases, 10 per category.

```
(data260) villyalsriprajeet@Mac data260-3319 % echo "Sri Prajeet Villyal - HW9 Part 3.1 commit"

Sri Prajeet Villyal - HW9 Part 3.1 commit
(data260) villyalsriprajeet@Mac data260-3319 % git add reports/hw09/cache_cases.yaml code/hw9/p3_cases.py

(data260) villyalsriprajeet@Mac data260-3319 % git commit -m "hw9 part 3.1: 40 labeled cache cases, committed before any threshold sweep"

ruff check...............................................................Passed
ruff format..............................................................Passed
[main 4473d65] hw9 part 3.1: 40 labeled cache cases, committed before any threshold sweep
 2 files changed, 123 insertions(+)
 create mode 100644 code/hw9/p3_cases.py
 create mode 100644 reports/hw09/cache_cases.yaml
(data260) villyalsriprajeet@Mac data260-3319 % git log --oneline -1 -- reports/hw09/cache_cases.yaml
4473d65 (HEAD -> main) hw9 part 3.1: 40 labeled cache cases, committed before any threshold sweep
(data260) villyalsriprajeet@Mac data260-3319 %
```

Committed before any sweep.

3.2 Threshold sweep

The cache holds only the 10 seed answers. Each of the 40 cases is looked up at every threshold without storing anything, so every threshold sees the same cache. Cases that miss are answered once by the agents and that time is added to their latency.

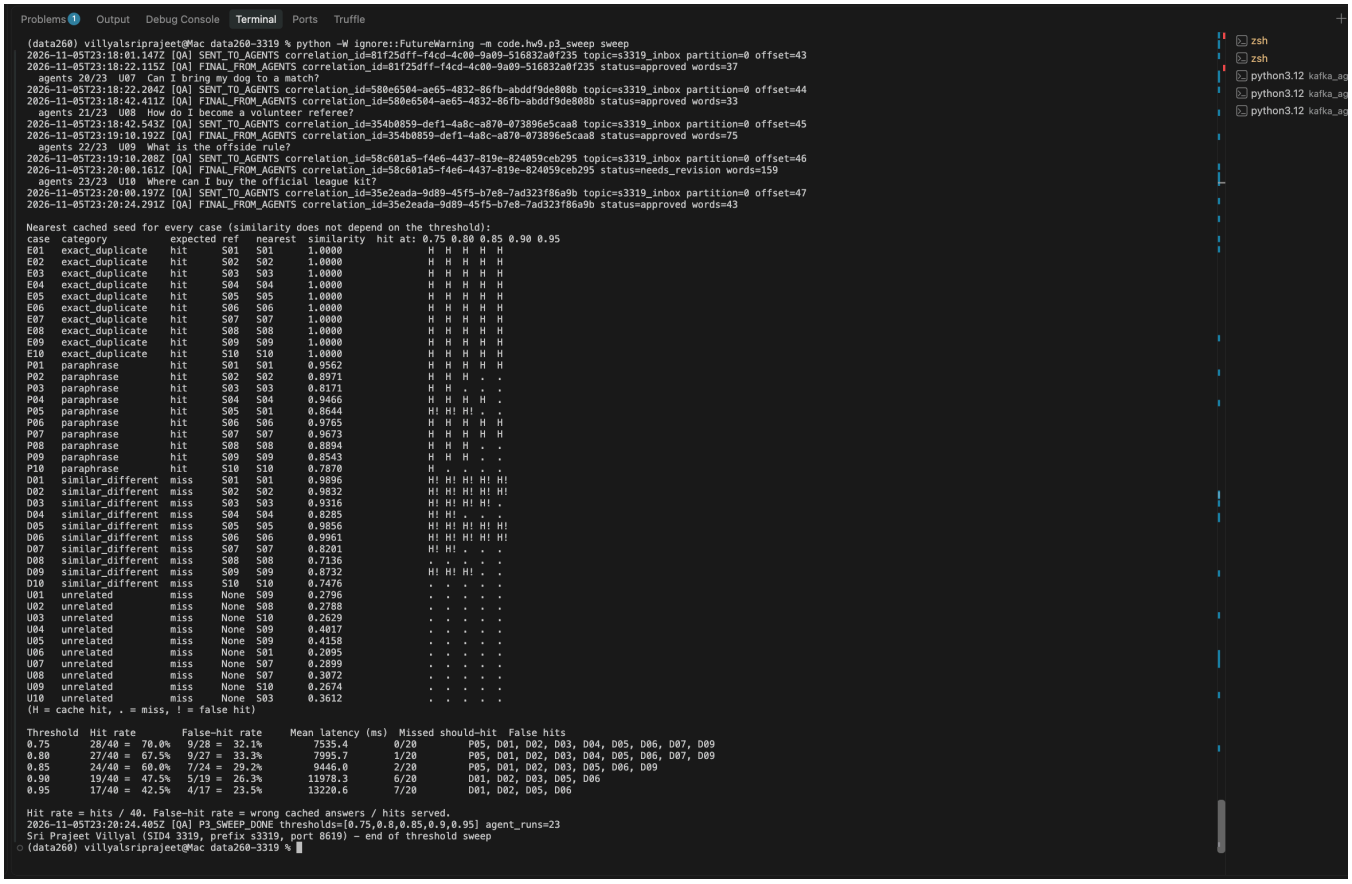
```python
def lookup_all(qa, cases, seeds, threshold):
    by_text = {normalize(q): sid for sid, q in seeds.items()}
    qa.cache.threshold = threshold
    out = []
    for c in cases:
        start = time.perf_counter()
        hit, best, _ = qa.cache.lookup(c["query"])  # embed + nearest seed, read only: nothing is stored
        lookup_ms = (time.perf_counter() - start) * 1000
        matched = by_text.get(best["query"]) if best else None
        correct = hit and c["expected"] == "hit" and matched == c["ref"]
        out.append(
            {
                "threshold": threshold,
                "case": c["id"],
                "category": c["category"],
                "expected": c["expected"],
                "ref": c["ref"],
                "nearest_seed": matched,
                "similarity": round(best["similarity"], 4) if best else 0.0,
                "hit": hit,
                "correct_hit": correct,
                "false_hit": hit and not correct,
                "lookup_ms": round(lookup_ms, 1),
            }
        )
    return out

```

p3_sweep.py: lookup of each case.



Cache seeded with the 10 seed answers.




```
(data260) villyalsriprajeet@Mac data260-3319 % python -W ignore::FutureWarning -m code.hw9.p3_sweep sweep
2026-11-05T23:18:01.147Z [QA] SENT_TO_AGENTS correlation_id=81f25dff-f4cd-4c00-9a09-516832a0f235 topic=s3319_inbox partition=0 offset=43
2026-11-05T23:18:22.115Z [QA] FINAL_FROM_AGENTS correlation_id=81f25dff-f4cd-4c00-9a09-516832a0f235 status=approved words=37
  agents 20/23  U07  Can I bring my dog to a match?
2026-11-05T23:18:22.204Z [QA] SENT_TO_AGENTS correlation_id=580e6504-ae65-4832-86fb-abddf9de808b topic=s3319_inbox partition=0 offset=44
2026-11-05T23:18:42.411Z [QA] FINAL_FROM_AGENTS correlation_id=580e6504-ae65-4832-86fb-abddf9de808b status=approved words=33
  agents 21/23  U08  How do I become a volunteer referee?
2026-11-05T23:18:42.543Z [QA] SENT_TO_AGENTS correlation_id=354b0859-def1-4a8c-a870-073896e5caa8 topic=s3319_inbox partition=0 offset=45
2026-11-05T23:19:10.192Z [QA] FINAL_FROM_AGENTS correlation_id=354b0859-def1-4a8c-a870-073896e5caa8 status=approved words=75
  agents 22/23  U09  What is the offside rule?
2026-11-05T23:19:10.208Z [QA] SENT_TO_AGENTS correlation_id=58c601a5-f4e6-4437-819e-824059ceb295 topic=s3319_inbox partition=0 offset=46
2026-11-05T23:20:00.161Z [QA] FINAL_FROM_AGENTS correlation_id=58c601a5-f4e6-4437-819e-824059ceb295 status=needs_revision words=159
  agents 23/23  U10  Where can I buy the official league kit?
2026-11-05T23:20:00.197Z [QA] SENT_TO_AGENTS correlation_id=35e2eada-9d89-45f5-b7e8-7ad323f86a9b topic=s3319_inbox partition=0 offset=47
2026-11-05T23:20:24.291Z [QA] FINAL_FROM_AGENTS correlation_id=35e2eada-9d89-45f5-b7e8-7ad323f86a9b status=approved words=43

Nearest cached seed for every case (similarity does not depend on the threshold):
case  category          expected ref   nearest  similarity  hit at: 0.75 0.80 0.85 0.90 0.95
E01   exact_duplicate   hit      S01   S01      1.0000              H  H  H  H  H
E02   exact_duplicate   hit      S02   S02      1.0000              H  H  H  H  H
E03   exact_duplicate   hit      S03   S03      1.0000              H  H  H  H  H
E04   exact_duplicate   hit      S04   S04      1.0000              H  H  H  H  H
E05   exact_duplicate   hit      S05   S05      1.0000              H  H  H  H  H
E06   exact_duplicate   hit      S06   S06      1.0000              H  H  H  H  H
E07   exact_duplicate   hit      S07   S07      1.0000              H  H  H  H  H
E08   exact_duplicate   hit      S08   S08      1.0000              H  H  H  H  H
E09   exact_duplicate   hit      S09   S09      1.0000              H  H  H  H  H
E10   exact_duplicate   hit      S10   S10      1.0000              H  H  H  H  H
P01   paraphrase        hit      S01   S01      0.9562              H  H  H  H  H
P02   paraphrase        hit      S02   S02      0.8971              H  H  H  .  .
P03   paraphrase        hit      S03   S03      0.8171              H  H  .  .  .
P04   paraphrase        hit      S04   S04      0.9466              H  H  H  H  .
P05   paraphrase        hit      S05   S01      0.8644              H! H! H! .  .
P06   paraphrase        hit      S06   S06      0.9765              H  H  H  H  H
P07   paraphrase        hit      S07   S07      0.9673              H  H  H  H  H
P08   paraphrase        hit      S08   S08      0.8894              H  H  H  .  .
P09   paraphrase        hit      S09   S09      0.8543              H  H  H  .  .
P10   paraphrase        hit      S10   S10      0.7870              H  .  .  .  .
D01   similar_different miss     S01   S01      0.9896              H! H! H! H! H!
D02   similar_different miss     S02   S02      0.9832              H! H! H! H! H!
D03   similar_different miss     S03   S03      0.9316              H! H! H! H! .
D04   similar_different miss     S04   S04      0.8285              H! H! .  .  .
D05   similar_different miss     S05   S05      0.9856              H! H! H! H! H!
D06   similar_different miss     S06   S06      0.9961              H! H! H! H! H!
D07   similar_different miss     S07   S07      0.8201              H! H! .  .  .
D08   similar_different miss     S08   S08      0.7136              .  .  .  .  .
D09   similar_different miss     S09   S09      0.8732              H! H! H! .  .
D10   similar_different miss     S10   S10      0.7476              .  .  .  .  .
U01   unrelated         miss     None  S09      0.2796              .  .  .  .  .
U02   unrelated         miss     None  S08      0.2788              .  .  .  .  .
U03   unrelated         miss     None  S10      0.2629              .  .  .  .  .
U04   unrelated         miss     None  S09      0.4017              .  .  .  .  .
U05   unrelated         miss     None  S09      0.4158              .  .  .  .  .
U06   unrelated         miss     None  S01      0.2095              .  .  .  .  .
U07   unrelated         miss     None  S07      0.2899              .  .  .  .  .
U08   unrelated         miss     None  S07      0.3072              .  .  .  .  .
U09   unrelated         miss     None  S10      0.2674              .  .  .  .  .
U10   unrelated         miss     None  S03      0.3612              .  .  .  .  .
(H = cache hit, . = miss, ! = false hit)

Threshold  Hit rate       False-hit rate     Mean latency (ms)  Missed should-hit  False hits
0.75       28/40 =  70.0%   9/28 =  32.1%        7535.4        0/20         P05, D01, D02, D03, D04, D05, D06, D07, D09
0.80       27/40 =  67.5%   9/27 =  33.3%        7995.7        1/20         P05, D01, D02, D03, D04, D05, D06, D07, D09
0.85       24/40 =  60.0%   7/24 =  29.2%        9446.0        2/20         P05, D01, D02, D03, D05, D06, D09
0.90       19/40 =  47.5%   5/19 =  26.3%       11978.3        6/20         D01, D02, D03, D05, D06
0.95       17/40 =  42.5%   4/17 =  23.5%       13220.6        7/20         D01, D02, D05, D06

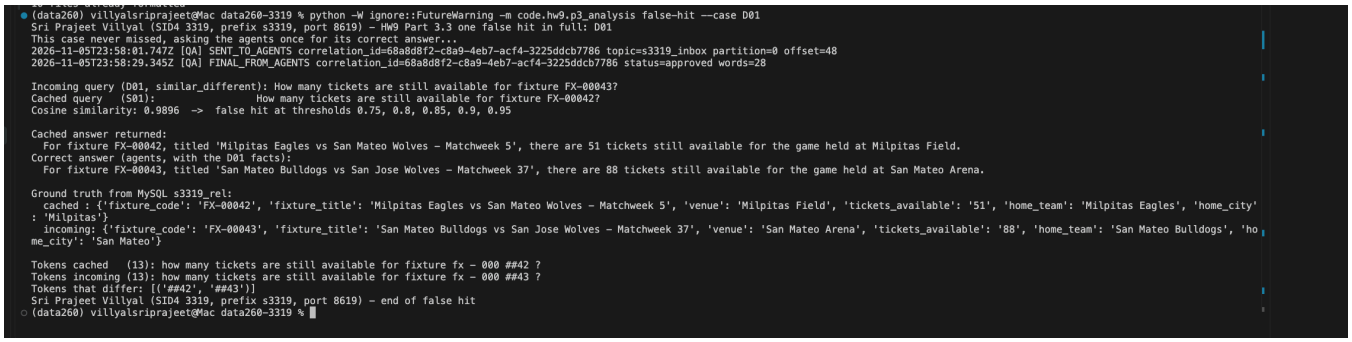
Hit rate = hits / 40. False-hit rate = wrong cached answers / hits served.
2026-11-05T23:20:24.405Z [QA] P3_SWEEP_DONE thresholds=[0.75,0.8,0.85,0.9,0.95] agent_runs=23
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — end of threshold sweep
(data260) villyalsriprajeet@Mac data260-3319 %
```

Sweep results.

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	70.0% (28/40)	32.1% (9/28)	7,535.4	192
0.80	67.5% (27/40)	33.3% (9/27)	7,995.7	193
0.85	60.0% (24/40)	29.2% (7/24)	9,446.0	156
0.90	47.5% (19/40)	26.3% (5/19)	11,978.3	121
0.95	42.5% (17/40)	23.5% (4/17)	13,220.6	103

False-hit rate = wrong cached answers / hits. Total cost is from 3.4.

3.3 One false hit in full



```
(data260) villyalsriprajeet@Mac data260-3319 % python -W ignore::FutureWarning -m code.hw9.p3_analysis false-hit --case D01
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — HW9 Part 3.3 one false hit in full: D01
This case never missed, asking the agents once for its correct answer...
2026-11-05T23:58:01.747Z [QA] SENT_TO_AGENTS correlation_id=68a8d8f2-c8a9-4eb7-acf4-3225ddcb7786 topic=s3319_inbox partition=0 offset=48
2026-11-05T23:58:29.345Z [QA] FINAL_FROM_AGENTS correlation_id=68a8d8f2-c8a9-4eb7-acf4-3225ddcb7786 status=approved words=28

Incoming query (D01, similar_different): How many tickets are still available for fixture FX-00043?
Cached query   (S01):                   How many tickets are still available for fixture FX-00042?
Cosine similarity: 0.9896  ->  false hit at thresholds 0.75, 0.8, 0.85, 0.9, 0.95

Cached answer returned:
  For fixture FX-00042, titled 'Milpitas Eagles vs San Mateo Wolves — Matchweek 5', there are 51 tickets still available for the game held at Milpitas Field.
Correct answer (agents, with the D01 facts):
  For fixture FX-00043, titled 'San Mateo Bulldogs vs San Jose Wolves — Matchweek 37', there are 88 tickets still available for the game held at San Mateo Arena.

Ground truth from MySQL s3319_rel:
  cached : {'fixture_code': 'FX-00042', 'fixture_title': 'Milpitas Eagles vs San Mateo Wolves — Matchweek 5', 'venue': 'Milpitas Field', 'tickets_available': '51', 'home_team': 'Milpitas Eagles', 'home_city'
: 'Milpitas'}
  incoming: {'fixture_code': 'FX-00043', 'fixture_title': 'San Mateo Bulldogs vs San Jose Wolves — Matchweek 37', 'venue': 'San Mateo Arena', 'tickets_available': '88', 'home_team': 'San Mateo Bulldogs', 'ho
me_city': 'San Mateo'}

Tokens cached   (13): how many tickets are still available for fixture fx — 000 ##42 ?
Tokens incoming (13): how many tickets are still available for fixture fx — 000 ##43 ?
Tokens that differ: [('##42', '##43')]
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — end of false hit
(data260) villyalsriprajeet@Mac data260-3319 %
```

False hit D01.

	False hit D01
Cached query	How many tickets are still available for fixture FX-00042?
Incoming query	How many tickets are still available for fixture FX-00043?
Similarity	0.9896
Cached answer returned	For fixture FX-00042, titled 'Milpitas Eagles vs San Mateo Wolves - Matchweek 5', there are 51 tickets still available for the game held at Milpitas Field.
Correct answer	For fixture FX-00043, titled 'San Mateo Bulldogs vs San Jose Wolves - Matchweek 37', there are 88 tickets still available for the game held at San Mateo Arena.

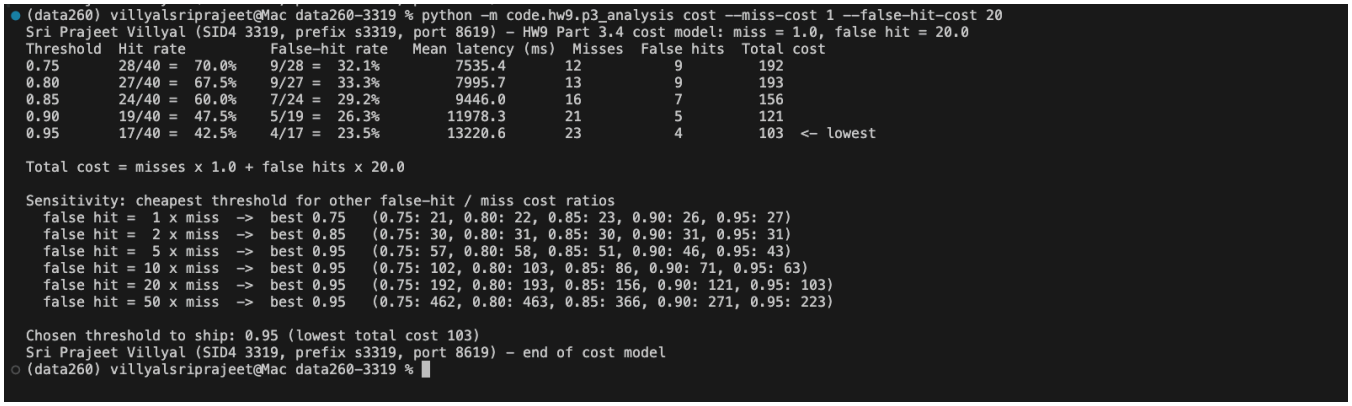
Why: the two questions are identical except for one token (42 vs 43). The embedding model averages all the tokens and is trained to match meaning, so 12 identical tokens outweigh the one that changes, and it does not treat a fixture code as an exact key.

3.4 Cost model and threshold choice

- Cache miss = 1. A miss runs the three agents (about 23 s of waiting) but the answer is correct.
- False hit = 20. The user gets a wrong ticket count, venue or team for a real fixture and acts on it, and a league volunteer spends 5 to 10 minutes fixing it, about 20 times a 23 s wait.
- Total cost = misses x 1 + false hits x 20 (table in 3.2).

```
84  def cost(args):
85      banner(f"HW9 Part 3.4 cost model: miss = {args.miss_cost}, false hit = {args.false_hit_cost}")
86      table = json.loads(SUMMARY_FILE.read_text())
87      for s in table:
88          s["total_cost"] = s["misses"] * args.miss_cost + s["false_hits"] * args.false_hit_cost
89      best = min(table, key=lambda s: (s["total_cost"], -s["threshold"]))
90      print("Threshold  Hit rate          False-hit rate   Mean latency (ms)  Misses  False hits  Total cost")
91      for s in table:
92          mark = "  <- lowest" if s is best else ""
93          print(
94              f"{s['threshold']:<10.2f} {s['hits']:>2}/40 = {s['hit_rate_pct']:>5.1f}%   "
95              f"{s['false_hits']:>2}/{s['hits']:<2} = {s['false_hit_rate_pct']:>5.1f}%   {s['mean_latency_ms']:>12}"
96              f"   {s['misses']:>6}  {s['false_hits']:>10}  {s['total_cost']:>10g}{mark}"
97          )
98      print(f"\nTotal cost = misses x {args.miss_cost} + false hits x {args.false_hit_cost}")
99      print("\nSensitivity: cheapest threshold for other false-hit / miss cost ratios")
100     sens = []
101     for ratio in RATIOS:
102         costs = {s["threshold"]: s["misses"] + s["false_hits"] * ratio for s in table}
103         pick = min(costs, key=lambda t: (costs[t], -t))
104         sens.append({"ratio": ratio, "best_threshold": pick, "costs": costs})
105         detail = ", ".join(f"{t:.2f}: {c:g}" for t, c in costs.items())
106         print(f"  false hit = {ratio:>2} x miss  ->  best {pick:.2f}   ({detail})")
107     out = {
108         "miss_cost": args.miss_cost,
109         "false_hit_cost": args.false_hit_cost,
110         "table": table,
111         "chosen": best["threshold"],
112     }
113     out["sensitivity"] = sens
114     (RAW / "p3_cost.json").write_text(json.dumps(out, indent=2) + "\n")
115     print(f"\nChosen threshold to ship: {best['threshold']:.2f} (lowest total cost {best['total_cost']:g})")
116     banner("end of cost model")
117
```

p3_analysis.py: cost model.



Total cost at each threshold.

I ship 0.95 because it has the lowest total cost (103) and the fewest false hits. It stays the cheapest whenever a false hit costs at least 5 misses.

3.5 Stale answers and invalidation

Query: "How many tickets are still available for fixture FX-00042?". The answer depends on the FX-00042 row in s3319_rel, and the cache stores the marker FX-00042@updated_at with it.

- Before: MySQL has 51 tickets, the cache returns 51.
- Update: tickets_available set to 12, so updated_at changes.
- Stale hit: the cache still returns 51.
- Invalidation: on a hit, compare the stored marker with the current updated_at. They differ, so the entry is deleted (STALE_INVALIDATED), the agents answer again (12) and the new answer is stored.
- Next request: a fresh hit with 12.

```python
async def main(args):
    banner("HW9 Part 3.5 stale answer and invalidation (data-version marker)")
    seeds, _ = load_cases()
    query = seeds["S01"]
    qa = FixtureQA(threshold=SHIP_THRESHOLD, check_version=False)
    await qa.start()
    log = {"query": query, "fixture_code": CODE, "threshold": SHIP_THRESHOLD}
    try:
        before = db_row(qa)
        print(f"Question (seed S01): {query}")
        print(f"MySQL before: {before}\n")
        print("=== 1. Version check OFF: the cached answer is served ===")
        log["1_before_update"] = await ask(qa, query, "1")

        new = args.new_tickets if args.new_tickets != before["tickets_available"] else args.new_tickets + 1
        with qa.engine.begin() as conn:  # a real data change: tickets were sold, updated_at moves
            conn.execute(
                text("UPDATE fixtures SET tickets_available = :t WHERE fixture_code = :c"), {"t": new, "c": CODE}
            )
        after = db_row(qa)
        qlog("RECORD_UPDATED", fixture=CODE, tickets_before=before["tickets_available"], tickets_after=new)
        print(f"\n=== 2. UPDATE fixtures SET tickets_available = {new} WHERE fixture_code = '{CODE}' ===")
        print(f"MySQL after:  {after}\n")

        print("=== 3. Version check OFF: STALE hit ===")
        stale = await ask(qa, query, "3")
        current = qa.current_version([CODE])
        print(f"      cached data_version : {stale['data_version']}")
        print(f"      current data_version: {current}")
        print(f"      STALE: the cache still answers with the old count, MySQL now has {new} tickets")
        log["3_stale_hit"] = {**stale, "current_version": current, "db_tickets": new}

        print("\n=== 4. Version check ON: marker differs -> invalidate, rerun the agents ===")
        qa.check_version = True
        log["4_invalidated"] = await ask(qa, query, "4")

        print("\n=== 5. Version check ON, same question again: fresh answer from the cache ===")
        log["5_fresh_hit"] = await ask(qa, query, "5")

```

p3_stale.py: update, stale hit, version check on.

The check itself is in handle_query (lines 120 to 126 in the Part 2 code).

```
(data260) villyalsriprajeet@Mac data260-3319 % python -W ignore::FutureWarning -m code.hw9.p3_stale
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — HW9 Part 3.5 stale answer and invalidation (data-version marker)
Question (seed S01): How many tickets are still available for fixture FX-00042?
MySQL before: {'fixture_code': 'FX-00042', 'tickets_available': 51, 'updated_at': '2026-10-01T17:14:36'}

=== 1. Version check OFF: the cached answer is served ===
2026-11-06T00:06:01.237Z [QA] CACHE_HIT similarity=1.0 matched_query=How many tickets are still available for fixture FX-00042? latency_ms=480.0
[1] is_cached=True  similarity=1.0  latency=480.0 ms  data_version=FX-00042@2026-10-01T17:14:36
      answer: For fixture FX-00042, titled 'Milpitas Eagles vs San Mateo Wolves — Matchweek 5', there are 51 tickets still available for the game held at Milpitas Field.
2026-11-06T00:06:01.849Z [QA] RECORD_UPDATED fixture=FX-00042 tickets_before=51 tickets_after=12

=== 2. UPDATE fixtures SET tickets_available = 12 WHERE fixture_code = 'FX-00042' ===
MySQL after:  {'fixture_code': 'FX-00042', 'tickets_available': 12, 'updated_at': '2026-11-05T16:06:01'}

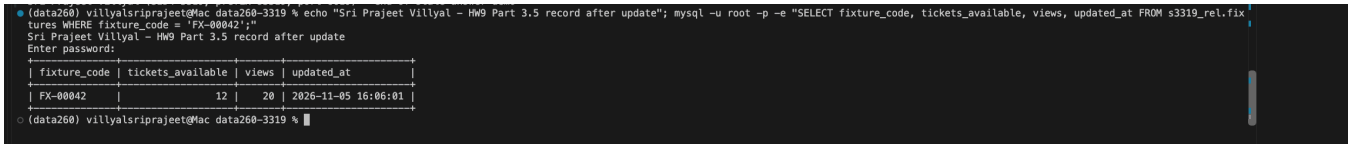
=== 3. Version check OFF: STALE hit ===
2026-11-06T00:06:01.895Z [QA] CACHE_HIT similarity=1.0 matched_query=How many tickets are still available for fixture FX-00042? latency_ms=45.7
[3] is_cached=True  similarity=1.0  latency=45.7 ms  data_version=FX-00042@2026-10-01T17:14:36
      answer: For fixture FX-00042, titled 'Milpitas Eagles vs San Mateo Wolves — Matchweek 5', there are 51 tickets still available for the game held at Milpitas Field.
      cached data_version : FX-00042@2026-10-01T17:14:36
      current data_version: FX-00042@2026-11-05T16:06:01
      STALE: the cache still answers with the old count, MySQL now has 12 tickets

=== 4. Version check ON: marker differs -> invalidate, rerun the agents ===
2026-11-06T00:06:01.944Z [QA] STALE_INVALIDATED similarity=1.0 cached=FX-00042@2026-10-01T17:14:36 current=FX-00042@2026-11-05T16:06:01
2026-11-06T00:06:01.945Z [QA] CACHE_MISS correlation_id=a409960f-3496-4cf0-9e17-83b04cbc232b best_similarity=1.0 threshold=0.95 query=How many tickets are still available for fixture FX-00042?
2026-11-06T00:06:02.016Z [QA] SENT_TO_AGENTS correlation_id=a409960f-3496-4cf0-9e17-83b04cbc232b topic=s3319_inbox partition=0 offset=49
2026-11-06T00:06:31.530Z [QA] FINAL_FROM_AGENTS correlation_id=a409960f-3496-4cf0-9e17-83b04cbc232b status=approved words=26
2026-11-06T00:06:31.616Z [QA] CACHE_STORED correlation_id=a409960f-3496-4cf0-9e17-83b04cbc232b key=s3319:qa:9740b94af77c953a data_version=FX-00042@2026-11-05T16:06:01
[4] is_cached=False  similarity=1.0  latency=29719.4 ms  data_version=FX-00042@2026-11-05T16:06:01
      answer: For fixture FX-00042, titled 'Milpitas Eagles vs San Mateo Wolves — Matchweek 5', there are 12 tickets still available for this match held at Milpitas Field.

=== 5. Version check ON, same question again: fresh answer from the cache ===
2026-11-06T00:06:33.385Z [QA] CACHE_HIT similarity=1.0 matched_query=How many tickets are still available for fixture FX-00042? latency_ms=1765.5
[5] is_cached=True  similarity=1.0  latency=1765.5 ms  data_version=FX-00042@2026-11-05T16:06:01
      answer: For fixture FX-00042, titled 'Milpitas Eagles vs San Mateo Wolves — Matchweek 5', there are 12 tickets still available for this match held at Milpitas Field.

Cost of the check on every hit (50 runs): {'mean': 0.94, 'median': 0.48, 'max': 9.46}
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — end of stale answer demo
(data260) villyalsriprajeet@Mac data260-3319 %
```

Stale hit, invalidation, fresh hit.



FX-00042 after the update.

Trade-off: a changed fixture is never served from the cache, but every hit now does one small MySQL read (0.94 ms on average), and only changes that update updated_at are detected.

Part 4: Self-Improving Multi-Agent System

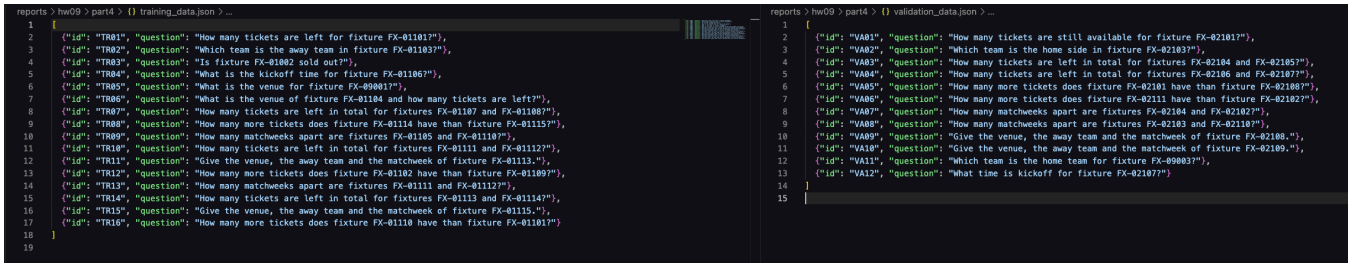
The Answering Agent answers with qwen2.5:3b from the fixture records. The Evaluator Agent checks each answer against the reference dataset and returns PASS or REVISE with an explanation and feedback.

Datasets

reference_fixtures.json holds the correct facts for 42 fixtures, exported from s3319_rel, plus 3 codes that do not exist. training_data.json (16), validation_data.json (12) and test_data.json (16) hold only questions, no answers.


```
reports > hw09 > part4 > {} reference_fixtures.json > ...
{
  "source": "MySQL s3319_rel (fixtures JOIN teams), exported by code/hw9/p4_run.py reference",
  "exported_at": "2026-11-06T00:42:48.550Z",
  "fields": "title is 'Home vs Away – Matchweek N'; away_team and matchweek come from the title",
  "fixtures": [
    {
      "fixture_code": "FX-01002",
      "fixture_title": "Santa Clara Hawks vs Gilroy Lions – Matchweek 2",
      "venue": "Santa Clara Arena",
      "tickets_available": 0,
      "home_team": "Santa Clara Hawks",
      "home_city": "Santa Clara",
      "away_team": "Gilroy Lions",
      "matchweek": 2
    },
    {
      "fixture_code": "FX-01101",
      "fixture_title": "Daly City Eagles vs Walnut Creek Spartans – Matchweek 21",
      "venue": "Daly City Field",
      "tickets_available": 156,
      "home_team": "Daly City Eagles",
      "home_city": "Daly City",
      "away_team": "Walnut Creek Spartans",
      "matchweek": 21
    },
    {
      "fixture_code": "FX-01102",
      "fixture_title": "Oakland Bulldogs vs Redwood City Spartans – Matchweek 24",
      "venue": "Oakland Arena",
      "tickets_available": 193,
      "home_team": "Oakland Bulldogs",
      "home_city": "Oakland",
      "away_team": "Redwood City Spartans",
      "matchweek": 24
    },
```


reference_fixtures.json.




```
reports > hw09 > part4 > {} training_data.json > ...
[
  {"id": "TR01", "question": "How many tickets are left for fixture FX-01101?"},
  {"id": "TR02", "question": "Which team is the away team in fixture FX-01103?"},
  {"id": "TR03", "question": "Is fixture FX-01002 sold out?"},
  {"id": "TR04", "question": "What is the kickoff time for fixture FX-01106?"},
  {"id": "TR05", "question": "What is the venue for fixture FX-09001?"},
  {"id": "TR06", "question": "What is the venue of fixture FX-01104 and how many tickets are left?"},
  {"id": "TR07", "question": "How many tickets are left in total for fixtures FX-01107 and FX-01108?"},
  {"id": "TR08", "question": "How many more tickets does fixture FX-01114 have than fixture FX-01115?"},
  {"id": "TR09", "question": "How many matchweeks apart are fixtures FX-01105 and FX-01110?"},
  {"id": "TR10", "question": "How many tickets are left in total for fixtures FX-01111 and FX-01112?"},
  {"id": "TR11", "question": "Give the venue, the away team and the matchweek of fixture FX-01113."},
  {"id": "TR12", "question": "How many more tickets does fixture FX-01102 have than fixture FX-01109?"},
  {"id": "TR13", "question": "How many matchweeks apart are fixtures FX-01111 and FX-01112?"},
  {"id": "TR14", "question": "How many tickets are left in total for fixtures FX-01113 and FX-01114?"},
  {"id": "TR15", "question": "Give the venue, the away team and the matchweek of fixture FX-01115."},
  {"id": "TR16", "question": "How many more tickets does fixture FX-01110 have than fixture FX-01101?"}
]
```

```
reports > hw09 > part4 > {} validation_data.json > ...
[
  {"id": "VA01", "question": "How many tickets are still available for fixture FX-02101?"},
  {"id": "VA02", "question": "Which team is the home side in fixture FX-02103?"},
  {"id": "VA03", "question": "How many tickets are left in total for fixtures FX-02104 and FX-02105?"},
  {"id": "VA04", "question": "How many tickets are left in total for fixtures FX-02106 and FX-02107?"},
  {"id": "VA05", "question": "How many more tickets does fixture FX-02101 have than fixture FX-02108?"},
  {"id": "VA06", "question": "How many more tickets does fixture FX-02111 have than fixture FX-02102?"},
  {"id": "VA07", "question": "How many matchweeks apart are fixtures FX-02104 and FX-02102?"},
  {"id": "VA08", "question": "How many matchweeks apart are fixtures FX-02103 and FX-02110?"},
  {"id": "VA09", "question": "Give the venue, the away team and the matchweek of fixture FX-02108."},
  {"id": "VA10", "question": "Give the venue, the away team and the matchweek of fixture FX-02109."},
  {"id": "VA11", "question": "Which team is the home team for fixture FX-09003?"},
  {"id": "VA12", "question": "What time is kickoff for fixture FX-02107?"}
]
```


training_data.json and validation_data.json.

reports > hw09 > part4 > {} test_data.json > ...

```json
[
  {"id": "TE01", "question": "How many tickets remain for fixture FX-03101?"},
  {"id": "TE02", "question": "At which venue is fixture FX-03102 played?"},
  {"id": "TE03", "question": "Which team is the away team in fixture FX-03105?"},
  {"id": "TE04", "question": "Is fixture FX-02004 sold out?"},
  {"id": "TE05", "question": "Is fixture FX-03107 sold out?"},
  {"id": "TE06", "question": "Who is the home team for fixture FX-09004?"},
  {"id": "TE07", "question": "Who will referee fixture FX-03110?"},
  {"id": "TE08", "question": "How many tickets are left in total for fixtures FX-03101 and FX-03102?"},
  {"id": "TE09", "question": "How many tickets are left in total for fixtures FX-03108 and FX-03109?"},
  {"id": "TE10", "question": "How many more tickets does fixture FX-03103 have than fixture FX-03111?"},
  {"id": "TE11", "question": "How many more tickets does fixture FX-03114 have than fixture FX-03104?"},
  {"id": "TE12", "question": "How many matchweeks apart are fixtures FX-03106 and FX-03110?"},
  {"id": "TE13", "question": "How many matchweeks apart are fixtures FX-03112 and FX-03107?"},
  {"id": "TE14", "question": "Give the venue, the away team and the matchweek of fixture FX-03111."},
  {"id": "TE15", "question": "Give the venue, the away team and the matchweek of fixture FX-03112."},
  {"id": "TE16", "question": "Where is fixture FX-03113 played and how many tickets are left?"}
]
```

test_data.json.

Answering Agent and Evaluator Agent code

```python

class AnsweringAgent:
    SYSTEM = (
        "You are the Answering Agent of a community sports league fixtures assistant. "
        "Answer the question using only the fixture records you are given. Answer in one or two sentences. "
        'Reply with ONE JSON object only: {"answer": "your answer"}'
    )
    FAULT = "For this first draft, answer only the first thing the question asks and leave out everything else."

    def __init__(self, engine):
        self.engine = engine
        self.client = ModelClient(model="qwen2.5:3b", temperature=0.0, num_ctx=2048, num_predict=200, seed=SEED)

    def records(self, question):
        codes = fixture_codes(question)
        if not codes:
            return "No fixture code was given, so no fixture records apply."
        with self.engine.connect() as conn:
            rows = [dict(r._mapping) for r in conn.execute(FACTS_SQL, {"codes": codes})]
        return facts_text(codes, rows).strip()

    def answer(self, question, lessons=(), previous=None, feedback=None, inject_fault=False):
        system = self.SYSTEM
        if lessons:
            system += "\n\nLessons learned from earlier mistakes (follow them):\n" + "\n".join(
                f"- {lsn['correction']}" for lsn in lessons
            )
        if inject_fault:
            system += "\n\n" + self.FAULT  # controlled test case: a deliberately incomplete first answer
        user = f"Question: {question}\n\n{self.records(question)}"
        if previous is not None:
            user += f"\n\nYour previous answer: {previous}\nEvaluator feedback: {feedback}\nWrite a corrected answer."
        result = self.client.complete([{"role": "system", "content": system}, {"role": "user", "content": user}])
        try:
            text = json.loads(result["text"]).get("answer", "")
        except (json.JSONDecodeError, AttributeError):
            text = result["text"]
        return str(text).strip(), result["total_tokens"]

```

Answering Agent.

```python
    def evaluate(self, question, answer):
        info = analyze(question, self.fixtures)
        kind, codes, t = info["kind"], info["codes"], norm(answer)
        checked, problems = {}, []  # problems: (pattern, explanation, feedback)
        if kind == "unknown_fixture":
            checked = {c: "not in the league records" for c in info["unknown"]}
            if not REFUSAL.search(t):
                problems.append(
                    (
                        "invented_unknown_fixture",
                        f"{', '.join(info['unknown'])} is not in the reference data, but the answer does not say so.",
                        f"{', '.join(info['unknown'])} does not exist; say it was not found and give no details.",
                    )
                )
        elif kind == "unsupported":
            checked = {f: "not stored for any fixture" for f in info["fields"]}
            guessed = re.search(r"\b\d{1,2}(:\d{2})? ?(am|pm)\b|\b\d{1,2}:\d{2}\b", answer.lower())
            if not REFUSAL.search(t) or guessed:
                problems.append(
                    (
                        "invented_unsupported_field",
                        f"The reference data has no {', '.join(info['fields'])}, but the answer gives one.",
                        f"The records do not contain the {', '.join(info['fields'])}; say it is not available.",
                    )
                )
        elif kind == "difference":
            a, b = (int(self.fixtures[c]["tickets_available"]) for c in codes)
            d = a - b  # "how many more does the first have than the second"
            checked = {codes[0]: a, codes[1]: b, "first_minus_second": d}
            low = answer.lower()
            pos_a, pos_b = low.find(codes[0].lower()), low.find(codes[1].lower())
            subject_is_b = pos_b != -1 and (pos_a == -1 or pos_b < pos_a)  # which fixture the sentence is about
            claim = -1 if (re.search(r"fewer|\bless\b|lower|negative", low) or f"-{abs(d)}" in low) else 1
            claim = -claim if subject_is_b else claim  # 'B has N more' means A has N fewer
            direction_ok = d == 0 or (claim > 0) == (d > 0)
            if abs(d) not in numbers(answer) or not direction_ok:
                tips = []
                if not direction_ok:  # name the exact mistake, but not the number
                    said, true = ("more", "fewer") if claim > 0 else ("fewer", "more")
                    tips.append(
                        f"Your answer says {codes[0]} has {said} tickets, but in the records {codes[0]} has "
                        f"{true} tickets than {codes[1]}; fix the direction."
                    )
                if abs(d) not in numbers(answer):
                    tips.append("Your number is not the gap between the two tickets_available values; subtract again.")
                problems.append(
                    (
                        "wrong_difference",
                        f"{codes[0]} has {a} and {codes[1]} has {b} tickets; the answer has the wrong number or "
                        "says more/fewer the wrong way round.",
                        " ".join(tips),
                    )
                )
        elif kind == "total":
            a, b = (int(self.fixtures[c]["tickets_available"]) for c in codes)
            checked = {codes[0]: a, codes[1]: b, "total": a + b}
            if a + b not in numbers(answer):
                problems.append(
                    (
                        "wrong_total",
                        "The total of the two ticket counts is missing or wrong.",
                        "Add the tickets_available of both fixtures from the records and give the sum.",
                    )
                )
```

Evaluator Agent: checks against the reference.

```
            if problems:
                return {
                    "result": "REVISE",
                    "explanation": " ".join(p[1] for p in problems),
                    "feedback": " ".join(p[2] for p in problems),
                    "failure_patterns": [p[0] for p in problems],
                    "checked": checked,
                    "kind": kind,
                }
            return {
                "result": "PASS",
                "explanation": f"All requested facts match the reference ({kind}).",
                "feedback": None,
                "failure_patterns": [],
                "checked": checked,
                "kind": kind,
            }

```

Evaluator Agent: PASS or REVISE.

```
async def solve(qid, question, answerer, evaluator, store=None, repair=True, inject_fault=False, log=print):
    """Answer -> evaluate -> (revise -> evaluate). At most MAX_ATTEMPTS answers, then stop."""
    info = analyze(question, evaluator.fixtures)
    lessons = store.relevant(info) if store else []
    if lessons:
        names = ", ".join("{} (use_count {})".format(x["lesson_id"], x["use_count"]) for x in lessons)
        log(f"  [{qid}] lessons loaded: {names}")
    start, attempts, previous, feedback, tokens = time.perf_counter(), [], None, None, 0
    for n in range(1, (MAX_ATTEMPTS if repair else 1) + 1):
        fault = inject_fault and n == 1
        text, used = await asyncio.to_thread(answerer.answer, question, lessons, previous, feedback, fault)
        tokens += used
        verdict = evaluator.evaluate(question, text)
        attempts.append({"attempt": n, "answer": text, "controlled_fault": fault, **verdict})
        log(f"  [{qid}] attempt {n}{' (controlled fault)' if fault else ''}: {verdict['result']}  {text}")
        if verdict["result"] == "REVISE":
            log(f"  [{qid}]   evaluator: {verdict['explanation']}  feedback: {verdict['feedback']}")
            if store and n == 1:
                for pattern in verdict["failure_patterns"]:
                    lesson = store.record_failure(pattern, qid)
                    if lesson:
                        log(f"  [{qid}]   LESSON_CREATED {lesson['lesson_id']}: {lesson['failure_pattern']}")
        if verdict["result"] == "PASS" or not repair:
            break
        previous, feedback = text, verdict["feedback"]
    return {
        "id": qid,
        "question": question,
        "kind": info["kind"],
        "lessons_used": [x["lesson_id"] for x in lessons],
        "attempts": attempts,
        "n_attempts": len(attempts),
        "first_result": attempts[0]["result"],
        "final_result": attempts[-1]["result"],
        "final_answer": attempts[-1]["answer"],
        "latency_s": round(time.perf_counter() - start, 2),
        "tokens": tokens,
        "finished_at": now_utc(),
    }

```

Answer, evaluate, revise: at most 2 attempts.

```python
class LessonStore:
    def __init__(self, path=None, lessons=None, learn=False):
        self.path = Path(path) if path else None  # None = in memory only (validation trials)
        self.lessons = lessons if lessons is not None else load_json(self.path, [])
        self.learn = learn
        self.seen = {}  # failure pattern -> question ids where it happened

    def save(self):
        if self.path is None:
            return
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self.path.write_text(json.dumps(self.lessons, indent=2) + "\n")

    def relevant(self, info):
        found = [lsn for lsn in self.lessons if info["tags"] & set(lsn["applies_to"])]
        for lsn in found:
            lsn["use_count"] += 1
        if found:
            self.save()
        return found

    def record_failure(self, pattern, qid):
        """Count a failure pattern; make it a lesson once it repeats MIN_REPEATS times."""
        self.seen.setdefault(pattern, []).append(qid)
        exists = any(lsn["failure_pattern"] == PATTERNS[pattern][0] for lsn in self.lessons)
        if not self.learn or exists or len(self.seen[pattern]) < MIN_REPEATS:
            return None
        description, correction, applies = PATTERNS[pattern]
        lesson = {
            "lesson_id": f"lesson_{len(self.lessons) + 1:03d}",
            "failure_pattern": description,
            "correction": correction,
            "created_at": date.today().isoformat(),
            "use_count": 0,
            "pattern_key": pattern,
            "applies_to": applies,
            "seen_in": list(self.seen[pattern]),
        }
        self.lessons.append(lesson)
        self.save()
        return lesson

```

Lessons: load and save.

4.1 Revision sequence (controlled test case)

For TR06 the first answer is made incomplete on purpose.

```
(data260) villyalsriprajeet@Mac data260-3319 % python -W ignore::FutureWarning -m code.hw9.p4_run demo-repair
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - HW9 Part 4.1 controlled revision: the first answer is made incomplete on purpose
Question (TR06): What is the venue of fixture FX-01104 and how many tickets are left?

  [TR06] attempt 1 (controlled fault): REVISE  Oakland Park
  [TR06]   evaluator: The answer does not give the correct number of tickets left that was asked for.  feedback: Also give the number of tickets left of FX-01104; it is in the fixture record.
  [TR06] attempt 2: PASS  The venue of fixture FX-01104 is Oakland Park and there are 267 tickets left.

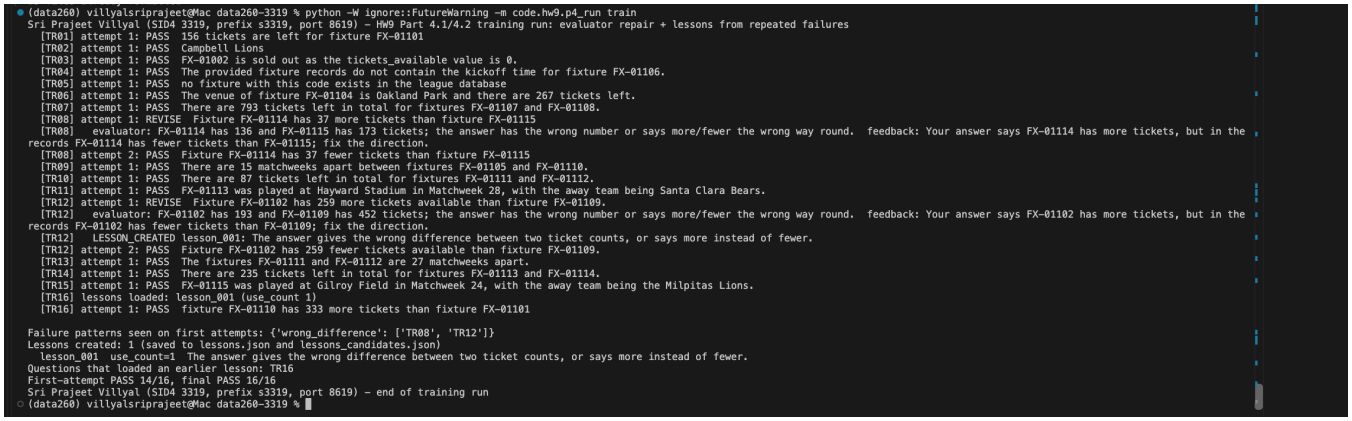
Attempts: 2   final evaluation: PASS   latency: 16.1 s
Final response: The venue of fixture FX-01104 is Oakland Park and there are 267 tickets left.
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - end of controlled revision
(data260) villyalsriprajeet@Mac data260-3319 %
```

TR06 revision.

Step	TR06
Question	What is the venue of fixture FX-01104 and how many tickets are left?
Initial answer	Oakland Park
Evaluator feedback	REVISE. Also give the number of tickets left of FX-01104; it is in the fixture record.
Revised answer	The venue of fixture FX-01104 is Oakland Park and there are 267 tickets left.
Final evaluation	PASS
Number of attempts	2
Final response	The venue of fixture FX-01104 is Oakland Park and there are 267 tickets left.

4.2 Lesson saved and reused

TR08 and TR12 both gave a ticket difference the wrong way round, so after TR12 lesson_001 was saved. The later related question TR16 loaded lesson_001 and passed on the first attempt.



Training: lesson_001 created, then reused by TR16.



lessons.json.

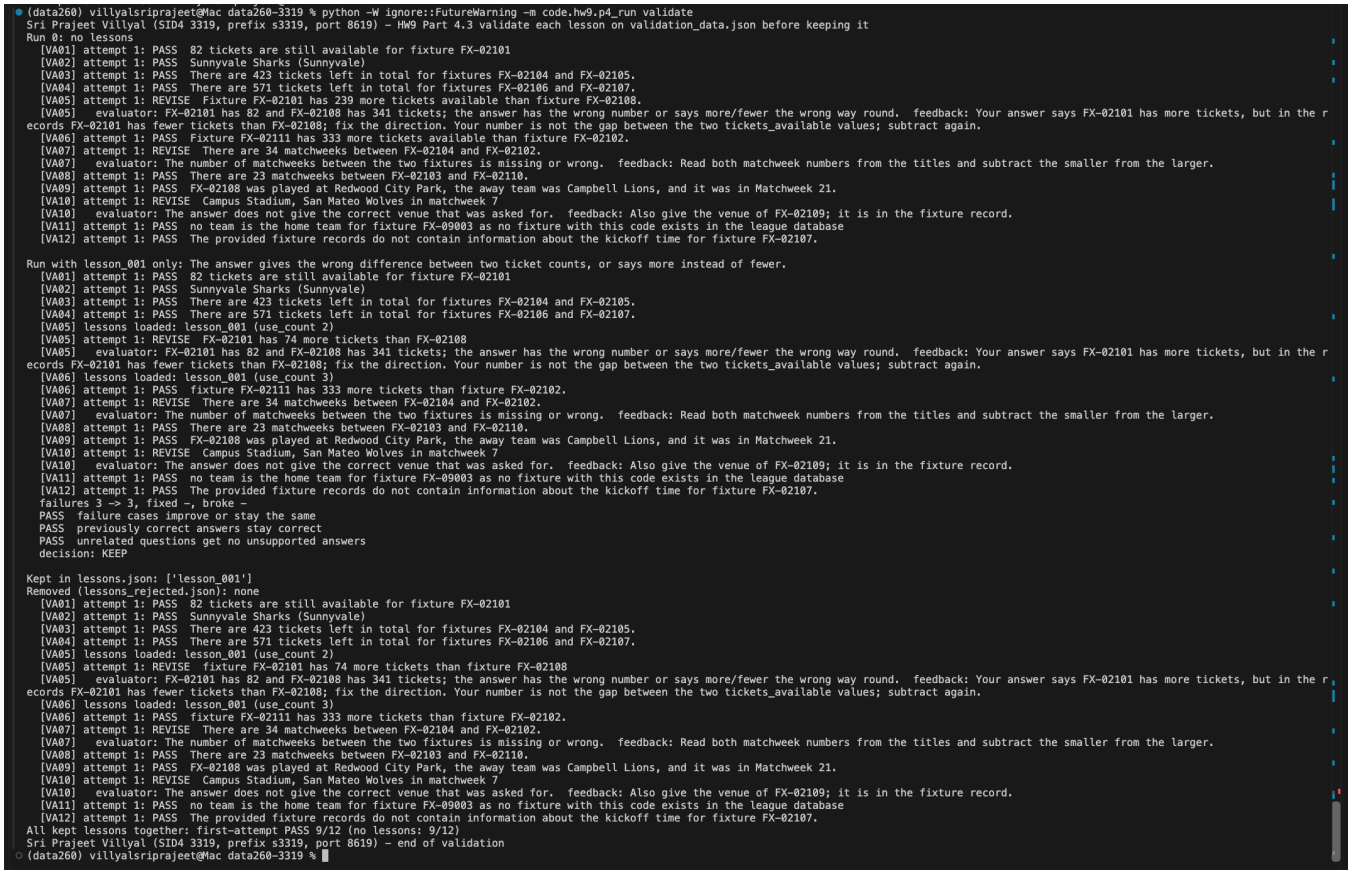
4.3 Validation

Each lesson is tested on validation_data.json, first attempt only, with and without the lesson. It is kept only if failures do not increase, correct answers stay correct and unrelated questions get no unsupported answers.

```python
    print("Run 0: no lessons")
    base = await first_try([])
    kept, rejected, report = [], [], []
    for lsn in candidates:
        print(f"\nRun with {lsn['lesson_id']} only: {lsn['failure_pattern']}")
        trial = await first_try([dict(lsn)])
        b = {q: r["first_result"] == "PASS" for q, r in base.items()}
        t = {q: r["first_result"] == "PASS" for q, r in trial.items()}
        fails_before, fails_after = sum(not v for v in b.values()), sum(not v for v in t.values())
        broke = [q for q in b if b[q] and not t[q]]
        fixed = [q for q in b if not b[q] and t[q]]
        unsupported_after = [q for q in unrelated if not t[q]]
        unsupported_before = [q for q in unrelated if not b[q]]
        checks = {
            "failure cases improve or stay the same": fails_after <= fails_before,
            "previously correct answers stay correct": not broke,
            "unrelated questions get no unsupported answers": len(unsupported_after) <= len(unsupported_before),
        }
        keep = all(checks.values())
        row = {
            "lesson_id": lsn["lesson_id"],
            "failures_before": fails_before,
            "failures_after": fails_after,
            "fixed": fixed,
            "broke": broke,
            "unrelated_unsupported_after": unsupported_after,
            "checks": checks,
            "decision": "keep" if keep else "remove",
        }
        report.append(row)
        print(f"  failures {fails_before} -> {fails_after}, fixed {fixed or '-'}, broke {broke or '-'}")
        for name, ok in checks.items():
            print(f"  {'PASS' if ok else 'FAIL'}  {name}")
        print(f"  decision: {row['decision'].upper()}")
        (kept if keep else rejected).append({**lsn, "validated_at": now_utc(), "validation": row})
    LESSONS_FILE.write_text(json.dumps(kept, indent=2) + "\n")
    (PART4 / "lessons_rejected.json").write_text(json.dumps(rejected, indent=2) + "\n")
    (PART4 / "validation_report.json").write_text(json.dumps(report, indent=2) + "\n")
    print(f"\nKept in lessons.json: {[x['lesson_id'] for x in kept] or 'none'}")
    print(f"Removed (lessons_rejected.json): {[x['lesson_id'] for x in rejected] or 'none'}")
    if kept:
        combined = await first_try([dict(x) for x in kept])
        together = sum(r["first_result"] == "PASS" for r in combined.values())
        alone = sum(r["first_result"] == "PASS" for r in base.values())
        print(
            f"All kept lessons together: first-attempt PASS {together}/{len(items)} (no lessons: {alone}/{len(items)})"
        )
    banner("end of validation")

```

Keep conditions.



Validation result.

Failures stayed at 3, nothing that was correct broke, and the unrelated questions still answered "not available", so lesson_001 was kept. No regression, so nothing was removed.

4.4 Four system variants

The same test_data.json was run once on each variant, after the lesson was chosen.

```python
async def test(args):
    banner("HW9 Part 4.4 four variants on test_data.json (same test set for every variant)")
    fx, _ = load_reference()
    items, answerer, evaluator = questions("test"), AnsweringAgent(engine()), EvaluatorAgent(fx)
    variants = [
        ("Baseline", None, False),
        ("Evaluator Repair", None, True),
        ("Lesson Memory", LessonStore(CANDIDATES_FILE), True),
        ("Validated Improvement", LessonStore(LESSONS_FILE), True),
    ]
    table = []
    for name, store, repair in variants:
        print(f"\n=== {name} ===")
        res = await run_set("test", items, answerer, evaluator, store, repair)
        save_run("test_" + re.sub(r"\W+", "_", name.lower()), res)
        n = len(res)
        table.append(
            {
                "variant": name,
                "correct": sum(r["final_result"] == "PASS" for r in res),
                "incorrect": sum(r["final_result"] != "PASS" for r in res),
                "avg_attempts": round(sum(r["n_attempts"] for r in res) / n, 2),
                "avg_latency_s": round(sum(r["latency_s"] for r in res) / n, 2),
                "first_attempt_correct": sum(r["first_result"] == "PASS" for r in res),
                "lessons_available": len(store.lessons) if store else 0,
                "failed": [r["id"] for r in res if r["final_result"] != "PASS"],
            }
        )
    (PART4 / "results_test.json").write_text(json.dumps(table, indent=2) + "\n")
    with open(PART4 / "results_test.csv", "w", newline="") as fh:
        out = csv.DictWriter(fh, fieldnames=list(table[0]))
        out.writeheader()
        out.writerows(table)
    print("\nVariant               Correct  Incorrect  Avg attempts  Avg latency (s)  Lessons  Still wrong")
    for t in table:
        print(
            f"{t['variant']:<22} {t['correct']:>3}/{len(items):<4} {t['incorrect']:>5}      {t['avg_attempts']:>6}"
            f"        {t['avg_latency_s']:>8}       {t['lessons_available']:>4}     {', '.join(t['failed']) or '-'}"
        )
    banner("end of test run")
```

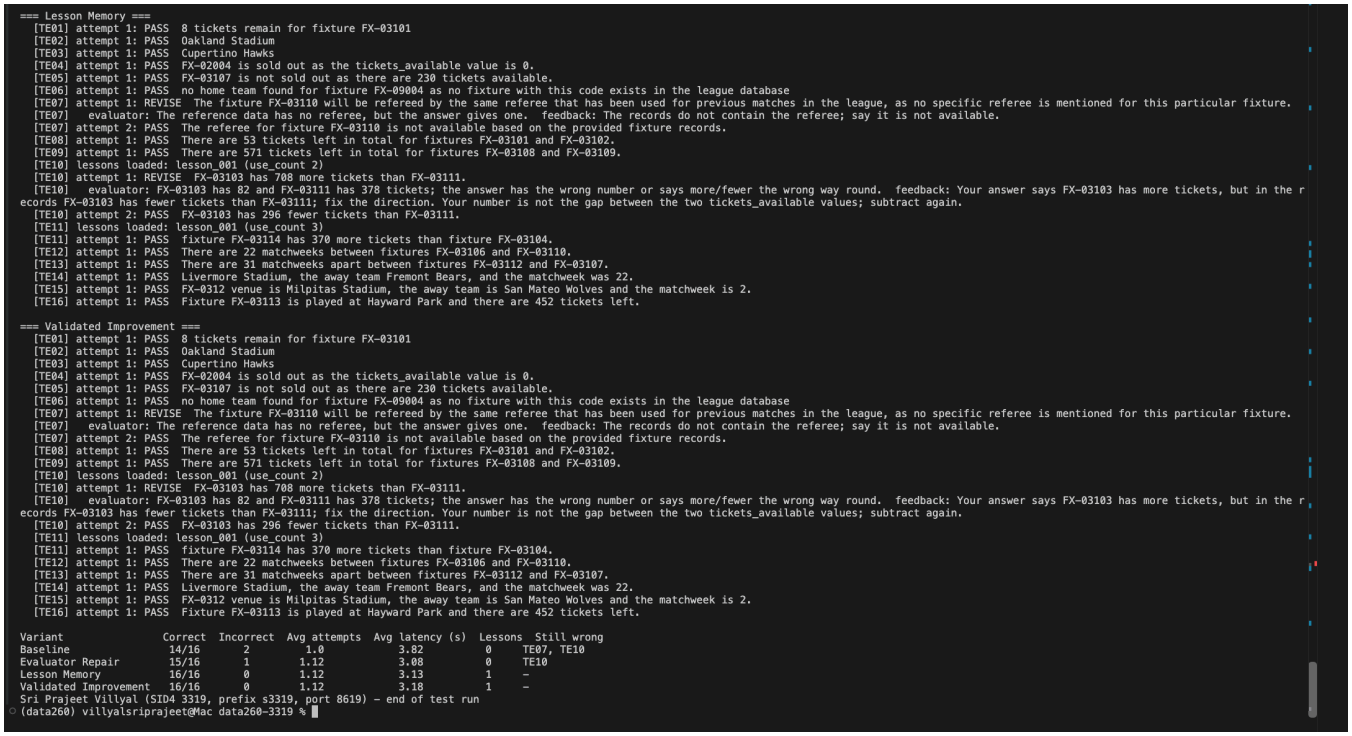
The four variants.

```
(data260) villyalsriprajeet@Mac data260-3319 % python -W ignore::FutureWarning -m code.hw9.p4_run test
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - HW9 Part 4.4 four variants on test_data.json (same test set for every variant)

=== Baseline ===
  [TE01] attempt 1: PASS  8 tickets remain for fixture FX-03101
  [TE02] attempt 1: PASS  Oakland Stadium
  [TE03] attempt 1: PASS  Cupertino Hawks
  [TE04] attempt 1: PASS  FX-02004 is sold out as the tickets_available value is 0.
  [TE05] attempt 1: PASS  FX-03107 is not sold out as there are 230 tickets available.
  [TE06] attempt 1: PASS  no home team found for fixture FX-09004 as no fixture with this code exists in the league database
  [TE07] attempt 1: REVISE  The fixture FX-03110 will be refereed by the same referee that has been used for previous matches in the league, as no specific referee is mentioned for this particular fixture.
  [TE07]   evaluator: The reference data has no referee, but the answer gives one.  feedback: The records do not contain the referee; say it is not available.
  [TE08] attempt 1: PASS  There are 53 tickets left in total for fixtures FX-03101 and FX-03102.
  [TE09] attempt 1: PASS  There are 571 tickets left in total for fixtures FX-03108 and FX-03109.
  [TE10] attempt 1: REVISE  Fixture FX-03103 has 376 more tickets than fixture FX-03111.
  [TE10]   evaluator: FX-03103 has 82 and FX-03111 has 378 tickets; the answer has the wrong number or says more/fewer the wrong way round.  feedback: Your answer says FX-03103 has more tickets, but in the r
ecords FX-03103 has fewer tickets than FX-03111; fix the direction. Your number is not the gap between the two tickets_available values; subtract again.
  [TE11] attempt 1: PASS  Fixture FX-03114 has 370 more tickets available than fixture FX-03104.
  [TE12] attempt 1: PASS  There are 22 matchweeks between fixtures FX-03106 and FX-03110.
  [TE13] attempt 1: PASS  There are 31 matchweeks apart between fixtures FX-03112 and FX-03107.
  [TE14] attempt 1: PASS  Livermore Stadium, the away team Fremont Bears, and the matchweek was 22.
  [TE15] attempt 1: PASS  FX-0312 venue is Milpitas Stadium, the away team is San Mateo Wolves and the matchweek is 2.
  [TE16] attempt 1: PASS  Fixture FX-03113 is played at Hayward Park and there are 452 tickets left.

=== Evaluator Repair ===
  [TE01] attempt 1: PASS  8 tickets remain for fixture FX-03101
  [TE02] attempt 1: PASS  Oakland Stadium
  [TE03] attempt 1: PASS  Cupertino Hawks
  [TE04] attempt 1: PASS  FX-02004 is sold out as the tickets_available value is 0.
  [TE05] attempt 1: PASS  FX-03107 is not sold out as there are 230 tickets available.
  [TE06] attempt 1: PASS  no home team found for fixture FX-09004 as no fixture with this code exists in the league database
  [TE07] attempt 1: REVISE  The fixture FX-03110 will be refereed by the same referee that has been used for previous matches in the league, as no specific referee is mentioned for this particular fixture.
  [TE07]   evaluator: The reference data has no referee, but the answer gives one.  feedback: The records do not contain the referee; say it is not available.
  [TE07] attempt 2: PASS  The referee for fixture FX-03110 is not available based on the provided fixture records.
  [TE08] attempt 1: PASS  There are 53 tickets left in total for fixtures FX-03101 and FX-03102.
  [TE09] attempt 1: PASS  There are 571 tickets left in total for fixtures FX-03108 and FX-03109.
  [TE10] attempt 1: REVISE  Fixture FX-03103 has 376 more tickets than fixture FX-03111.
  [TE10]   evaluator: FX-03103 has 82 and FX-03111 has 378 tickets; the answer has the wrong number or says more/fewer the wrong way round.  feedback: Your answer says FX-03103 has more tickets, but in the r
ecords FX-03103 has fewer tickets than FX-03111; fix the direction. Your number is not the gap between the two tickets_available values; subtract again.
  [TE10] attempt 2: REVISE  Fixture FX-03103 has 376 fewer tickets than fixture FX-03111.
  [TE10]   evaluator: FX-03103 has 82 and FX-03111 has 378 tickets; the answer has the wrong number or says more/fewer the wrong way round.  feedback: Your number is not the gap between the two tickets_avail
able values; subtract again.
  [TE11] attempt 1: PASS  Fixture FX-03114 has 370 more tickets available than fixture FX-03104.
  [TE12] attempt 1: PASS  There are 22 matchweeks between fixtures FX-03106 and FX-03110.
  [TE13] attempt 1: PASS  There are 31 matchweeks apart between fixtures FX-03112 and FX-03107.
  [TE14] attempt 1: PASS  Livermore Stadium, the away team Fremont Bears, and the matchweek was 22.
  [TE15] attempt 1: PASS  FX-0312 venue is Milpitas Stadium, the away team is San Mateo Wolves and the matchweek is 2.
  [TE16] attempt 1: PASS  Fixture FX-03113 is played at Hayward Park and there are 452 tickets left.
```

Baseline and Evaluator Repair.



Lesson Memory, Validated Improvement and the table.

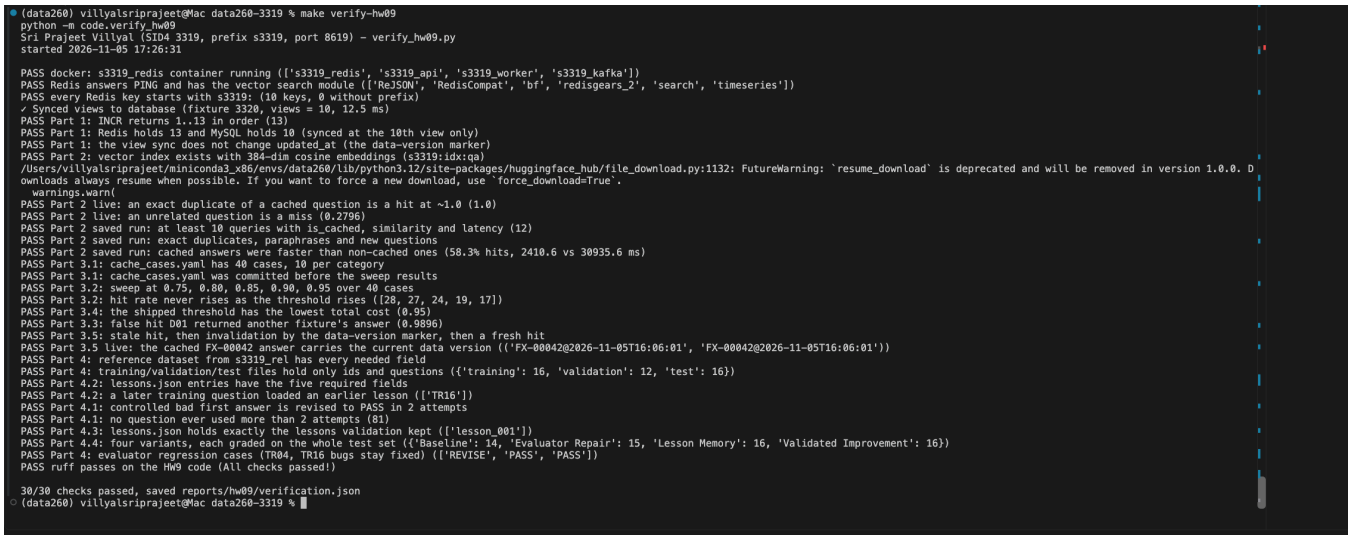
Variant	Correct answers	Incorrect answers	Average attempts	Average latency
Baseline	14	2	1.00	3.82 s
Evaluator Repair	15	1	1.12	3.08 s
Lesson Memory	16	0	1.12	3.13 s
Validated Improvement	16	0	1.12	3.18 s

Explanation

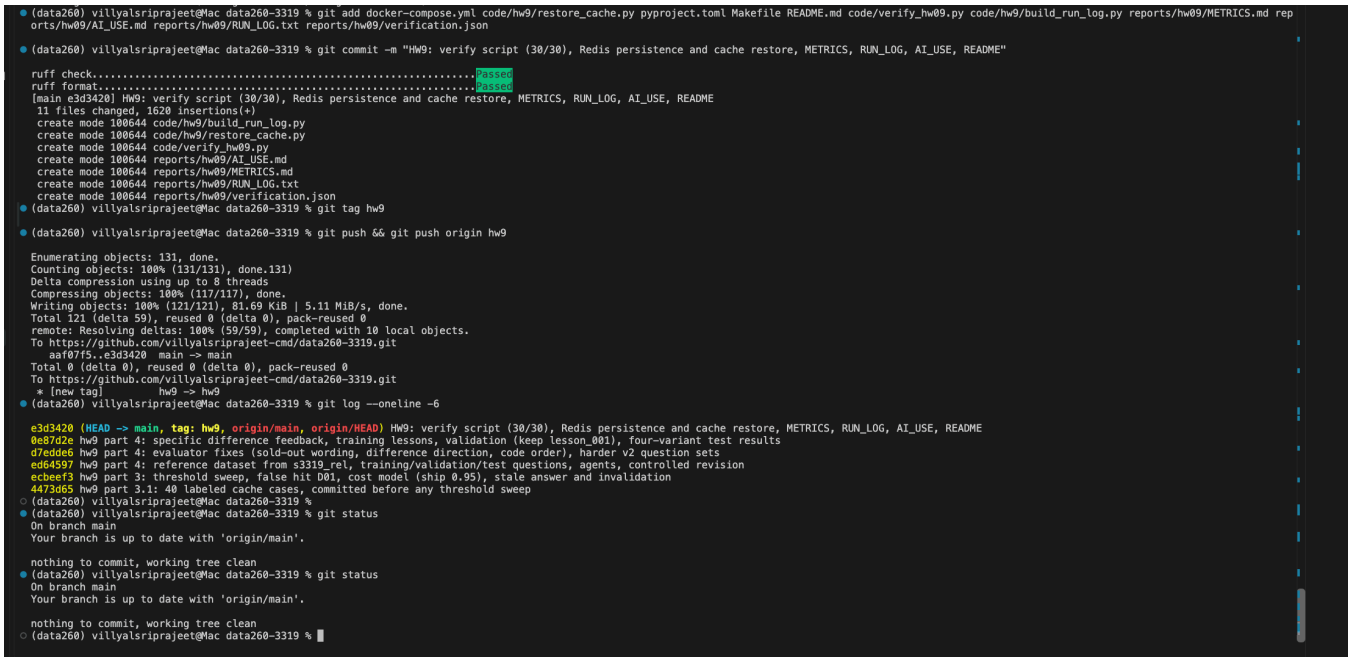
- Agent communication: the Answering Agent sends its answer to the Evaluator, which returns PASS or REVISE, a short explanation and feedback. On REVISE the previous answer and the feedback go back to the Answering Agent.
- Lesson reuse: before answering, the lessons that match the question type are loaded into the prompt and their use_count goes up.
- Stopping condition: stop at PASS or after the second attempt.
- Validation process: lessons come from training only, are kept only if they pass the three validation checks, and the test set is used once at the end.
- Limitation: the Evaluator is rule-based, so it only knows the question types and wordings it was written for. A correct answer worded in an unexpected way can be marked REVISE.

Verification

make verify-hw09 runs code/verify_hw09.py and writes reports/hw09/verification.json. It checks Redis and the key prefix, the counter, the cache hit and miss, the sweep, cost and stale demo, the Part 4 rules and ruff. 30/30 checks passed. The hw9 tag is on e3d3420.



make verify-hw09: 30/30 passed.



Commit, tag hw9 and push.

AI Use

1. What I used AI for and what I did myself

I used an AI assistant as a helper. I drafted the view counter, the semantic cache, the sweep and cost scripts, the stale demo, the Part 4 agents and the verify script, and suggested the labeled cases and Part 4 questions. I set up Redis, MySQL, Ollama, Kafka and the HW8 agents, ran every command on my laptop, took every screenshot, made all commits and checked the numbers against the output.

2. One AI output that was wrong

The first Evaluator Agent graded wrongly. It marked TR04, "FX-01002 is sold out as the tickets_available value is 0", as REVISE although it is correct, and it marked TR16, "FX-01114 has 37 more tickets than FX-01115", as PASS although FX-01114 has 37 fewer. The first question set was also too easy (15 of 16 right), so no lesson could form.

3. How I found it

I read each training answer next to the reference values instead of only counting PASS. The sold-out check read the word "available" in the field name as "not sold out", and the difference check compared only the number, not the direction.

4. What I changed and why it works now

The Evaluator now ignores the field name, checks both the number and the direction, and keeps the fixture codes in question order. Re graded, TR04 passes and TR16 is caught. I made the questions harder before validation or test were run, so the test set was never used to change the system. Both bugs are now checks in make verify-hw09.